FUJINET ENGINEERING SERIES

The Network Protocol Handbook

Every protocol the FujiNet `N:` device speaks — file servers, web services, sockets, shells, mail, and calendars — with the complete devicespec grammar, every value and error code, and worked examples for each.

With chapters for Atari BASIC · Applesoft BASIC · ADAM SmartBASIC · C with fujinet-lib · IntyBASIC · FujiNet NOS

28 schemes · 25 protocol adapters
One devicespec · one lifecycle · every machine

Revision 2 · September 2026
The FujiNet Project

About this handbook

Behind the FujiNet's `N:` device sits a library of **protocol adapters** — one for each way a vintage computer might want to reach the modern world. A TNFS file server, a web API, a raw TCP socket, an SSH shell, a Gmail mailbox, a Google calendar: each is a scheme in a devicespec string, and each obeys the same open–status–read/write–close lifecycle. This handbook documents every one of them — 28 scheme strings across 25 adapters — at the protocol level: the devicespec each expects, the open modes and special commands it honors, every constant and limit, the exact errors it returns, and a complete worked example.

The protocol chapters are host-neutral: they speak in aux values and command bytes, the vocabulary shared by every FujiNet bus. The final part turns that vocabulary into six dialects — Atari BASIC, Applesoft BASIC with the BASIC.SYSTEM extension, ADAM SmartBASIC 1.x, C with fujinet-lib, IntyBASIC on the Intellivision, and the FujiNet Network Operating System — each a focused digest of the full programmer's guide for that machine, with a pointer to the guide for the rest.

Every scheme string, aux value, command byte, constant, buffer size, and error code in this book was transcribed from the live project sources listed below, not from secondary documentation. Where the implementation has a rough edge or a deliberate gap, the text says so plainly.

Canonical sources

fujinet-firmware	`lib/network-protocol` — the protocol adapters (commits `c026d9a12` + `07e53c764`, Sep 2026)
fujinet-lib	v4.11.2 — `fujinet-network.h`, the C API used in the examples
fujinet-nhandler	the Atari `N:` handler and NOS; the Apple II BASIC.SYSTEM extension
smartbasic-1.x	the ADAM SmartBASIC 1.x FujiNet statements
fujinet-manuals	the per-platform programmer's guides digested in Part VI

Contents

PART I

The Network Device Model

One devicespec grammar, one four-beat lifecycle, one error table — the contract every protocol adapter honors and every host machine speaks. Read this part first; every chapter after it assumes this vocabulary.

CHAPTER 1

Introduction

To the computer on your desk, the FujiNet's network device is one more peripheral: a device you open with a name, read and write like a file, and close when you are done. The name is called a **devicespec**, and it is where all the power hides:

```
N:HTTPS://fujinet.online/hello.txt
```

Everything before the first colon addresses a device on your machine's peripheral bus. Everything after it is a URL, and the URL's scheme — `HTTPS` here — selects a **protocol adapter** inside the FujiNet firmware. There are 25 of those adapters, answering to 28 scheme strings, and together they are the subject of this book: file servers and web services, raw sockets and login shells, mailboxes and calendars, even the machine's own SD card and a CP/M emulator, each one reachable through the same little string.

1.1 One lifecycle for everything

Every protocol, from TNFS to Gmail, is driven by the same four beats:

1. **Open** — hand the FujiNet a devicespec, an access mode (aux1), and a translation mode (aux2). The firmware parses the URL, constructs the adapter for its scheme, and connects.
2. **Status** — ask how many bytes are waiting, whether the connection is still up, and the current error code. Status is cheap and you will call it often; several adapters do real work on the first status call.
3. **Read and write** — move bytes. Reading drains what the protocol has buffered; writing stages bytes and pushes them out.
4. **Close** — flush anything unwritten, tear the connection down, free the buffers.

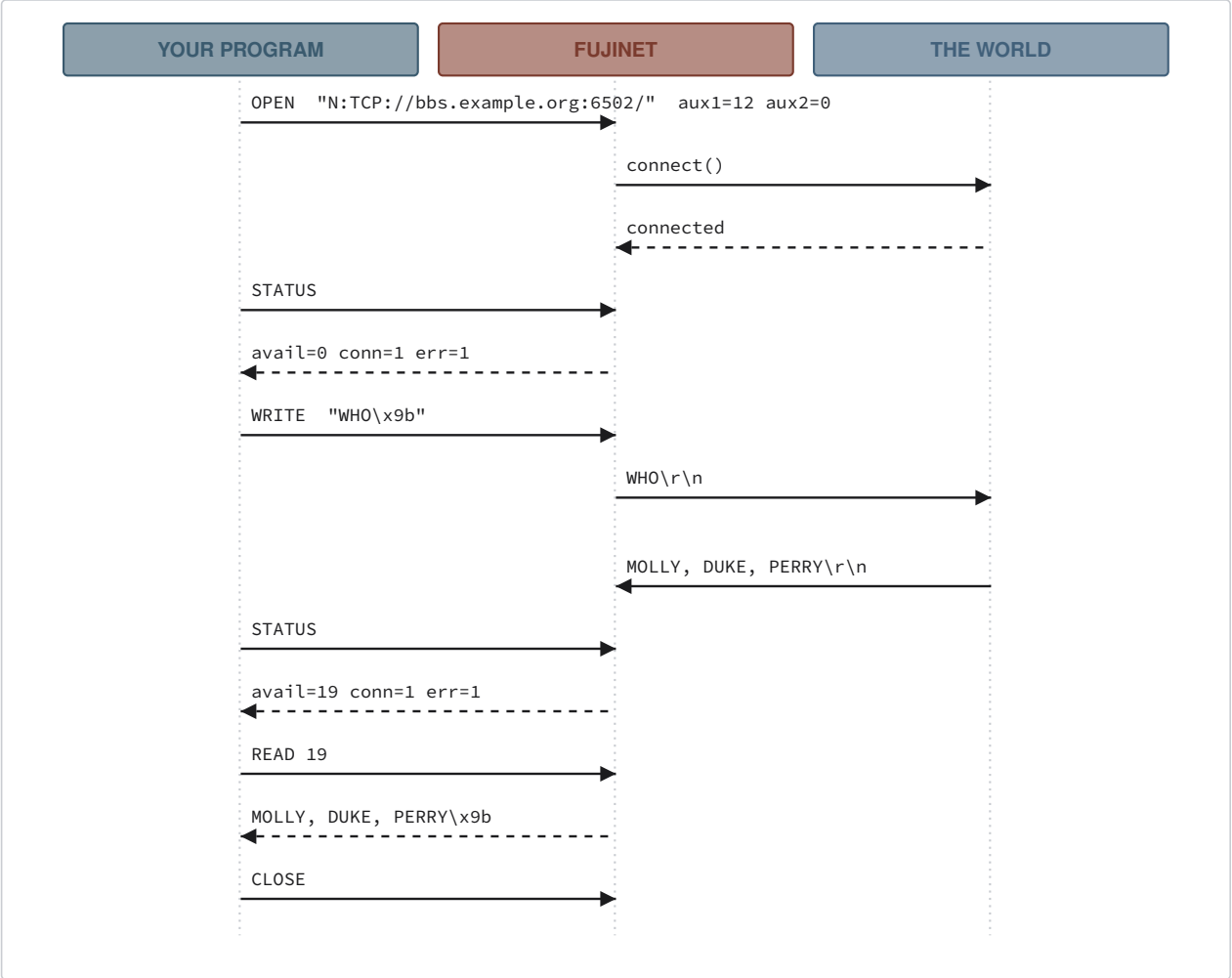

Figure 1. The four-beat lifecycle, here over TCP from an Atari. The FujiNet folds the network's CR/LF line endings into the machine's native end-of-line on the way through — that is the translation mode at work.

The beats wear different clothes on different machines — `OPEN #1,12,0` in Atari BASIC, `network_open()` in C, a mailbox transaction on the Intellivision — but the aux values, the command bytes, and the error codes underneath are identical, because they all land in the same firmware. That is why this handbook can be one book and not six.

1.2 The scheme table

The devicespec's scheme is uppercased and matched against this table (`ProtocolParser.cpp`). Anything else fails the open with error 165, invalid devicespec.

Scheme	Adapter class	What it reaches
`TNFS`	`NetworkProtocolTNFS`	TNFS file servers — the retro-native network filesystem
`SD`	`NetworkProtocolSD`	the SD card in the FujiNet itself
`SMB`	`NetworkProtocolSMB`	Windows / Samba file shares
`NFS`	`NetworkProtocolNFS`	UNIX NFSv3 exports
`FTP`	`NetworkProtocolFTP`	FTP servers (anonymous)
`SFTP`	`NetworkProtocolSFTP`	files over SSH
`HTTP` , `HTTPS`	`NetworkProtocolHTTP`	web servers, REST APIs, WebDAV shares

Scheme	Adapter class	What it reaches
`S3`	`NetworkProtocolS3`	Amazon S3 and compatible object stores
`GDRIVE`	`NetworkProtocolGDRIVE`	Google Drive
`ONEDRIVE`	`NetworkProtocolONEDRIVE`	Microsoft OneDrive
`TCP`	`NetworkProtocolTCP`	raw TCP sockets, client or listening server
`UDP`	`NetworkProtocolUDP`	UDP datagrams
`TELNET`	`NetworkProtocolTELNET`	Telnet with option negotiation
`SSH`	`NetworkProtocolSSH`	an interactive SSH shell
`WS` , `WSS`	`NetworkProtocolWS` / `WSS`	WebSocket connections, plain or TLS
`SSH.KEYGEN`	`NetworkProtocolSSHKeygen`	generate the FujiNet's SSH key pair
`SSH.COPYID`	`NetworkProtocolSSHCopyId`	install that key on a server
`CLIPBOARD`	`NetworkProtocolClipboard`	the FujiNet's clipboard and its history
`CPM`	`NetworkProtocolCPM`	a CP/M 2.2 machine running inside the FujiNet
`GMAIL`	`NetworkProtocolGMAIL`	a Gmail mailbox — read, compose, reply
`IMAPS`	`NetworkProtocolIMAPS`	any IMAP mailbox over TLS, read-only
`GCAL`	`NetworkProtocolGCAL`	Google Calendar — read, compose, edit
`ICAL` , `WEBCAL` , `ICALH`	`NetworkProtocolICAL`	published iCalendar feeds, read-only
`TEST`	`NetworkProtocolTest`	a fixed test pattern; the skeleton for new adapters

Table 1. All 28 scheme strings and the 25 adapters they construct.

1.3 The shape of the library

The adapters share three family trees, and the family a protocol belongs to tells you most of what it can do before you read its chapter:

Filesystem protocols derive from `NetworkProtocolFS`. They know the difference between a file and a directory, they can list directories in half a dozen formats, and they support some subset of rename, delete, mkdir, rmdir, lock, unlock, and seek. Part II.

Stream protocols derive from `NetworkProtocol` directly. Bytes in, bytes out, no files — sockets, shells, WebSockets. Part III, with the session utilities in Part IV.

Mail and calendar protocols derive from `NetworkProtocolMailbox` or `NetworkProtocolCalendar`, which map folders, messages, events, and views onto the path portion of the devicespec and render both human-readable and raw binary listings. Part V.

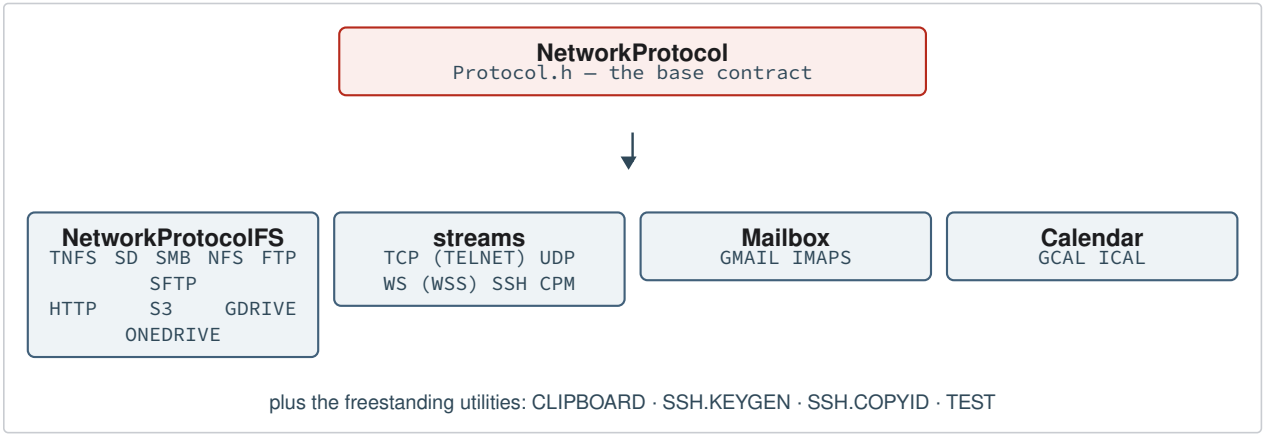

Figure 2. The class families in `lib/network-protocol`. TELNET subclasses TCP; WSS subclasses WS; the mail and calendar bases do the heavy lifting for their providers.

One architectural fact is worth fixing in your mind now: **the adapter lives in the FujiNet, not in your computer.** Your machine sends small commands over its peripheral bus; the ESP32 (or the fujinet-pc process) holds the buffers, speaks TLS, parses the JSON, and walks the OAuth dance. A 1979 machine with 8 KB of RAM reads Gmail because the hard part happens on the other side of the bus cable.

1.4 How each chapter is laid out

Every protocol chapter in Parts II–V opens with a synopsis card — scheme, class, family, default port, credential source, and a one-line capability summary — and then covers, in order: what the protocol is and when you would use it; the devicespec it expects, including every query parameter; a reference of every value it accepts (open modes, special commands, constants, and limits); its complete error mapping; and worked examples. The examples come in two forms: a **transaction trace**, written in the host-neutral vocabulary of this part, and a complete C program against fujinet-lib, the one API that runs on every FujiNet platform. Part VI then shows how to say the same things in each machine's own language.

> **NOTE**
> This handbook documents the firmware as it is, at the commit named in the colophon. Where an adapter has a rough edge — a stubbed operation that reports success, an error code that cannot occur — the chapter says so, and Appendix E collects the full list.

CHAPTER 2

The Devicespec

Every conversation with the network device begins with a devicespec. This chapter is the grammar for that string; learn it once and every protocol chapter becomes a set of footnotes to it.

2.1 The grammar

```
N[unit]:SCHEME://user:password@host:port/path?query
```

Reading left to right:

`N[unit]:` the device name your computer's bus uses, with an optional unit digit — `N1:` through `N8:` on most platforms, and a bare `N:` meaning `N1:`. The unit selects one of several independent **channels**, each with its own protocol instance, buffers, and state. The FujiNet firmware strips this piece off before URL parsing; it never reaches the adapter.

`SCHEME` selects the protocol adapter, per the table in Chapter 1. Case does not matter — the firmware uppercases it before matching.

`user:password@` optional credentials, for the protocols that take them in the URL (Chapter 2.3).

`host:port` the server, with an optional port. Each protocol has its own default port — and a few, like UDP, refuse to guess.

`/path` what the protocol addresses within the server — a file path, a bucket and key, a mail folder, a calendar view. A few adapters give the path unusual meanings; their chapters say so.

`?query` optional `key=value` pairs separated by `&`, read by some protocols for options that fit nowhere else — a terminal type, an S3 region, a date range.

The firmware parses the URL portion with its `PeoplesUrlParser`, which exposes exactly the fields above. Two consequences are worth knowing. First, an unparseable URL fails the open with error 165 before any protocol code runs. Second, a **valid** URL whose scheme is unknown fails with error 144 — so a misspelled scheme and a malformed URL report differently.

> **TIP**
> Devicespecs are usually typed on machines that shout. Every scheme accepts uppercase, and the filesystem adapters go further: SMB rewrites its scheme to the lowercase form its library wants, HTTP lowercases before connecting, and SMB will even lowercase an all-caps username and password for you (Chapter 10). Type naturally for your machine; the firmware meets you halfway.

2.2 Channels and units

The unit digit selects a channel, and the channels are genuinely independent: you can hold a TCP session on `N1:` while listing an SMB share on `N2:` and polling a calendar on `N3:`. How many channels you get, and how they are named, is a property of your machine's bus, not of the protocols:

Platform	Channels	Named as
Atari (SIO)	8	`N1:` – `N8:`, devices `$71` – `$78`; bare `N:` is `N1:`
Apple II (SmartPort)	8 units	unit numbers on one network device; the BASIC extension exposes channels 1–15 of its own

Platform	Channels	Named as
Coleco ADAM (AdamNet)	2 devices	AdamNet ids `$09` / `$0A`; SmartBASIC channels 1–15 map onto EOS character devices 9+
Intellivision (mailbox)	8	mailbox `DEVICE` byte `$71` – `$78`

Table 2. Channel counts are a bus property. The per-platform chapters in Part VI give each machine's exact story.

2.3 Credentials: three roads in

A protocol that needs a login can receive it three ways, and the order matters:

1. **In the URL** — `N:SMB://molly:secret@den-pc/atari/`. The most direct road, and for the SSH family the **shape** of the URL picks the authentication method: password present means password auth, absent means key auth.
2. **By command, before the open** — the username (`$FD`) and password (`$FE`) special commands stash strings on the channel; the next open hands them to the adapter. This is `USER` / `PASS` in NOS, `NLOGIN` in the BASIC extensions, `XIO 253` / `254` in Atari BASIC.
3. **From stored configuration** — the OAuth family (GDRIVE, GMAIL, GCAL, ONEDRIVE) and S3 read tokens and keys the web UI stored in the FujiNet's flash; nothing secret ever crosses the vintage bus at all.

> **PITFALL**
> The firmware only hands the stashed credentials to the adapter when the **username** is non-empty (`ProtocolParser.cpp:126`). A password set by itself, with no username, never reaches the protocol. If a server wants password-only login, send a dummy username too.

2.4 Prefixes and relative paths

The filesystem protocols support a working directory per channel: the chdir special (`$2C`) sets it, getcwd (`$30`) reads it back, and once set, a devicespec without a scheme is resolved against it. This is what NOS builds its `NCD` command on, and it is why `LOAD JUMPMAN.XEX` can work on a machine whose real file lives at `N1:TNFS://server/atari/games/JUMPMAN.XEX`. The prefix machinery lives in the per-bus device code, so the fine points — such as `..` handling — belong to your platform's programmer's guide; what is universal is that the **mounted URL** is the channel's state, held in the FujiNet, shared by everything on the machine that touches the channel.

CHAPTER 3

Open Modes and Translation

The open command carries two bytes that shape everything after it. The first — **aux1**, the access mode — says what you intend to do with the channel. The second — **aux2**, the translation mode — says what a line ending looks like to your machine. This chapter is the reference for both.

3.1 Access modes (aux1)

aux1	Name	Bits	Meaning
4	READ	`0100`	read an existing resource
6	DIRECTORY	`0110`	list a directory (filesystem protocols)
7	DIRECTORY_ALT	`0111`	directory, alternate flavor — treated as 6
8	WRITE	`1000`	create or truncate, then write
9	APPEND	`1001`	write, preserving existing content
12	READWRITE	`1100`	both directions at once — the mode for sockets
5, 13, 14	—	—	HTTP-specific: DELETE, POST, PUT-with-headers. Chapter 14 owns these.

Table 3. Access modes, from the `ACCESS_MODE` enum in `Protocol.h`. Not every protocol honors every mode; each chapter's reference section lists its subset.

The bit pattern is the old CIO convention: bit 2 is read, bit 3 is write, and 6 marks a directory. The three extra values are HTTP overloads — outside HTTP they are rejected. When a protocol receives a mode it does not support, you get error 146 (not implemented) or 135/131 (read-only / write-only), and the open fails cleanly.

3.2 Translation modes (aux2)

Vintage machines do not agree on what a line ending is. The Atari ends lines with `$9B`; the ADAM and Apple II use CR; CP/M and the modern internet like CR/LF; UNIX likes LF. The translation mode names **the network side's** convention, and the FujiNet converts to and from your machine's native end-of-line as bytes cross:

aux2	Name	The network peer ends lines with
0	NONE	nothing — bytes pass untouched (binary mode)
1	CR	`$0D`
2	LF	`$0A`
3	CR/LF	`$0D $0A`
4	PETSCII	— character-set translation for Commodore machines, applied on top of the byte stream (UTF-8 conversion both ways)

Table 4. Translation modes, from `netProtoTranslation_t` in `Protocol.h`.

On receive, the named network ending is folded into your machine's native end-of-line; on transmit, the native ending expands into the network form. The native ending itself is a property of the bus — the Atari device sets `$9B`, most others CR — so the same aux2 value means the right thing on every machine. On the Atari, three control codes ride along with the line endings: BEL, backspace, and tab are mapped to their ATASCII equivalents (`$FD`, `$7E`, `$7F`) and back.

> **IMPORTANT**
> Use mode 0 for anything binary — downloads, disk images, protocol frames. A translation mode applied to binary data will corrupt every byte that happens to look like a line ending. This is also NOS's `NTRANS` rule: set translation 0 before copying a binary.

Two subtleties from the firmware are worth recording. First, translation happens **exactly once per buffer fill**: the adapters are careful to translate freshly received bytes only, so a multi-byte native ending split across two reads cannot be mangled. Second, the buses keep a **sticky** translation value (the `$54` special — `NTRANS` in the extensions) that is OR-ed into aux2 on later opens — except for directory opens, where aux2 means something else entirely (a directory format, Chapter 7; a display width, Parts V) and the sticky value is deliberately left out. The special value `$FF` in the sticky slot forces mode 0 — an escape hatch for languages such as Action! that cannot pass a zero aux cleanly.

Separately from all of this, the `$4C` (set EOL) special can override the **native** end-of-line the translations target, per channel, at runtime — aux1 zero restores the platform default. You will rarely want it; it exists for terminal programs that need the network's bytes verbatim while keeping a translation mode on.

3.3 Line endings in generated text

Several protocols **generate** text rather than relay it: directory listings, status responses, mail indexes, calendar views. Those are composed inside the adapter with the platform's display line ending already applied (`$9B` on Atari, CR on ADAM and Apple II, CR/LF for serial machines), and the adapters force translation off for them so the endings are not translated a second time. That is why a directory listing looks right everywhere with no effort on your part — and why aux2 is free to mean "format" or "width" on those opens.

CHAPTER 4

Channel Modes: Protocol, JSON, and SGML

A freshly opened channel hands you the protocol's bytes as they come. That is **protocol mode**, and for file transfers and sockets it is all you need. But the modern network speaks structured text — JSON above all — and parsing JSON in 8-bit BASIC is nobody's idea of fun. So each channel has a **channel mode**, set by the `$FC` special:

aux2	Mode	Reads return
0	PROTOCOL	the protocol's own bytes (default)
1	JSON	nothing until you parse and query; then query results
2	SGML	text extracted from HTML/SGML markup

Table 5. Channel modes (`network_data.h`). The mode belongs to the channel, not the protocol — it works over HTTP, TCP, or anything else that can carry the text.

4.1 The JSON three-beat

Fetching a value out of a web API is always the same figure:

1. Open the URL normally (mode 4), then set channel mode JSON (`$FC` with aux2 = 1).
2. **Parse** (`$50` , mnemonic `P`) — the FujiNet reads the whole response body and builds the document tree in its own RAM.
3. **Query** (`$51` , mnemonic `Q`) — send a path like `/results/0/name` ; the next read returns just that value, terminated with your machine's line ending.

Query paths are slash-separated element names, with numbers selecting array elements. You can query as many times as you like against one parse. The BASIC extensions fold beat 1′s mode switch into their `NJSONPARSE` statement; Atari BASIC makes you issue `XIO 252` yourself; fujinet-lib gives you `network_json_parse()` and `network_json_query()` . Whatever the dialect, the firmware underneath is identical.

Two knobs exist for odd cases, both on the `$FB` (set JSON parameters) special: aux1 = 0 sets how query **results** are translated (aux2 = 0–2), and aux1 = 1 sets the line ending byte appended to them (aux2 = the byte). Most programs never touch either.

> **NOTE**
> The JSON and SGML machinery lives outside the protocol adapters, in `lib/fnjson` and `lib/fnsgml` , and is driven by the per-bus network device. That is why **any** protocol can carry JSON: the channel mode wraps the adapter rather than living inside it.

> **PITFALL**
> Two different specials have confusingly similar names. `$FC` — **channel mode** — selects protocol/JSON/SGML for the channel, as above. `$4D` — **HTTP channel mode** — is an HTTP-only special that switches the channel between the response body and the headers (Chapter 14). Atari BASIC spells them `XIO 252` and… nothing; `$4D` is one of the commands the N: handler cannot reach (Chapter 34).

CHAPTER 5

Status, Errors, and Interrupts

Status is the heartbeat of every network program. This chapter documents what the four status bytes mean, every error value the firmware can report, and the interrupt machinery that lets a program sleep instead of poll.

5.1 The four status bytes

A status command returns this structure, identically on every bus:

avail (lo)	avail (hi)	connected	error

avail a 16-bit little-endian count of bytes ready to read right now. For filesystem reads it counts down the remaining file (capped at 65534); for sockets it is what the peer has sent that you have not read.

connected nonzero while the underlying connection or resource is live. For a listening TCP socket it means "a client is waiting"; for a file being read it means "not exhausted yet"; UDP, which has no connections, always reports 1.

error the channel's current error code, from the table below. `1` is success; `136` is end-of-file; anything at or above 128 is trouble.

The idiomatic read loop on every platform is **status until avail is nonzero (or connected drops), read exactly avail (or your buffer's worth), repeat until error 136.** Chapter by chapter, this book only documents the departures from that loop.

Status also does quiet work: for most protocols, a status call with an empty receive buffer triggers the adapter to pull whatever the network has ready into its buffer, so `avail` reflects reality. A few adapters do their **first** real work on status — HTTP does not even send the request until the first status after open (Chapter 14) — and flag themselves so the firmware raises an interrupt to invite that first call.

5.2 The error table

Errors below 128 are Atari CIO conventions kept for familiarity; errors from 200 up are FujiNet's own network codes. Every adapter maps its library's errors into this one table — each chapter gives its exact mapping.

Code	Name	Meaning
1	SUCCESS	operation completed
131	WRITE_ONLY	read attempted on a write-only channel
132	INVALID_COMMAND	command invalid for this channel state
135	READ_ONLY	write attempted on a read-only channel
136	END_OF_FILE	no more data will arrive
138	GENERAL_TIMEOUT	the operation timed out
144	GENERAL	a fatal error with no more specific code
146	NOT_IMPLEMENTED	command not implemented for this protocol
151	FILE_EXISTS	the file or directory already exists
162	NO_SPACE_ON_DEVICE	no space left (or a write-buffer limit hit)
165	INVALID_DEVICESPEC	the devicespec could not be parsed or used
166	INVALID_POINT	seek to an invalid position
167	ACCESS_DENIED	authentication passed but permission refused

Code	Name	Meaning
170	FILE_NOT_FOUND	no such file, folder, message, or event
200	CONNECTION_REFUSED	the peer refused or the connect failed
201	NETWORK_UNREACHABLE	no route to the network
202	SOCKET_TIMEOUT	the socket timed out mid-operation
203	NETWORK_DOWN	the network interface is down
204	CONNECTION_RESET	the peer reset the connection
205	CONNECTION_ALREADY_IN_PROGRESS	a connect is already underway
206	ADDRESS_IN_USE	local address or port already in use
207	NOT_CONNECTED	no connection where one is required
208	SERVER_NOT_RUNNING	server-mode command with no listening socket
209	NO_CONNECTION_WAITING	accept called with no client waiting
210	SERVICE_NOT_AVAILABLE	the remote service is unavailable
211	CONNECTION_ABORTED	connection aborted (defined but never raised)
212	INVALID_USERNAME_OR_PASSWORD	authentication failed
213	COULD_NOT_PARSE_JSON	a JSON body would not parse
214	CLIENT_GENERAL	HTTP 4xx-class client error
215	SERVER_GENERAL	HTTP 5xx-class server error
255	COULD_NOT_ALLOCATE_BUFFERS	the FujiNet is out of memory

Table 6. Every network status code, from `status_error_codes.h`. This table is reproduced with per-platform notes in Appendix A.

For the socket protocols, operating-system errors are folded in by a shared map: connection refused becomes 200, unreachable 201, timed out 202, network down 203, reset 204, in-progress 205, address-in-use 206, and anything else 144. (On Windows builds of fujinet-pc, a would-block condition is misreported as 206 — a known rough edge, Appendix E.)

One more piece of bookkeeping matters to you: **a failed open destroys the adapter**, so the specific error it died with is latched by the firmware (`open_error` in `network_data.h`) and reported by the next status. You can always open, then status, and learn why the open failed.

5.3 Interrupts and polling

On buses with an interrupt line (the Atari's PROCEED), the FujiNet runs a timer that peeks at each open channel and asserts the interrupt when bytes are waiting or the connection has dropped — so a program can sit in its idle loop until the network has something to say. The `$5A` special (set interrupt rate) tunes the timer's period. Inside the timer the firmware calls the adapter's status with a flag set (`fromInterrupt`), and most adapters answer from cached state rather than touching the network — the exception is TCP, which is safe to poll for real. None of this changes what your program sees in the status bytes; it only changes when it is worth asking.

CHAPTER 6

Special Commands

Beyond open, close, read, write, and status, the network device accepts a family of **special** commands — the verbs behind XIO on the Atari, the N-statements in the BASIC extensions, and `network_ioctl()` in fujinet-lib. This chapter is the full table and the semantics the protocols share; each protocol's chapter lists which specials it honors.

6.1 The command table

Command bytes were chosen so that printable ones match their mnemonic letter — `'R'` reads, `'W'` writes, `'P'` parses. On the Atari the XIO number **is** the command byte, which makes this table double as the XIO reference.

Hex	Dec	Char	Name	Action
`$20`	32	—	RENAME	rename; devicespec carries `old,new` (Ch. 7)
`$21`	33	`!`	DELETE	delete a file or object
`$23`	35	`#`	LOCK	make read-only (chmod 444 where real)
`$24`	36	`$`	UNLOCK	make writable (chmod 644)
`$25`	37	`%`	SEEK	POINT — set the read position
`$26`	38	`&`	TELL	NOTE — report the read position
`$2A`	42	`*`	MKDIR	create a directory
`$2B`	43	`+`	RMDIR	remove a directory
`$2C`	44	`,`	CHDIR	set the channel's prefix / mount
`$30`	48	`0`	GETCWD	read the prefix back
`$41`	65	`A`	ACCEPT	TCP server: accept the waiting client
`$43`	67	`C`	CLOSE	close the channel
`$44`	68	`D`	SET DESTINATION	UDP: set where writes go
`$45`	69	`E`	GET ERROR	read the latched error
`$4C`	76	`L`	SET EOL	override the native end-of-line
`$4D`	77	`M`	HTTP CHANNEL MODE	switch body/headers (HTTP only)
`$4F`	79	`O`	OPEN	open a devicespec
`$50`	80	`P`	PARSE	JSON/SGML: parse the body
`$51`	81	`Q`	QUERY	JSON/SGML: set the query path
`$52`	82	`R`	READ	read bytes
`$53`	83	`S`	STATUS	the four status bytes
`$54`	84	`T`	SET TRANSLATION	sticky aux2 for later opens
`$57`	87	`W`	WRITE	write bytes
`$5A`	90	`Z`	SET INTERRUPT RATE	tune the status-poll timer
`$63`	99	`c`	CLOSE CLIENT	TCP server: drop the accepted client
`$72`	114	`r`	GET REMOTE	UDP: who spoke last (fujinet-pc only)
`$FA`	250	—	SET CHANNEL	SmartPort: select the network unit
`$FB`	251	—	SET JSON PARAMETERS	query translation / line ending
`$FC`	252	—	CHANNEL MODE	protocol / JSON / SGML

Hex	Dec	Char	Name	Action
`$FD`	253	—	USERNAME	stash a login for the next open
`$FE`	254	—	PASSWORD	stash a password for the next open
`$FF`	255	—	GET DSTATS VALUE	ask a command's data direction

Table 7. The network command set, from `fujiCommandID.h`. `$80` / `$81` also exist as alternate parse/query codes for the ComLynx bus, and `$3F` / `$E3` are the Atari high-speed-index pair.

The last entry deserves a sentence: `$FF` takes a command byte and answers with its data direction — no payload, payload to computer, or payload from computer. It exists so a generic handler (the Atari's NDEV is the customer) can relay a command it has never heard of, by asking the FujiNet which way the bytes will flow. That is how new specials reach old handlers without re-flashing anything — and why an unknown command reports 146 rather than hanging the bus.

6.2 Filesystem specials are one-shot

Rename, delete, lock, unlock, mkdir, and rmdir do not use an open channel. The firmware constructs a **fresh** adapter from the devicespec in the command's payload, performs the one operation, and destroys the adapter again. Two practical consequences: the specials work while the channel is closed, and the devicespec must be complete — scheme, host, and path — every time. Aimed at a protocol that has no filesystem (TCP, say), they fail with a transaction error; aimed at one whose operation is unimplemented, error 146 or a silent success, as each chapter documents.

6.3 Credentials, again

The `$FD` / `$FE` pair stores the username and password **on the channel**, to be handed to the next adapter constructed there — which includes the one-shot adapters above. The strings persist until replaced. And to repeat the pitfall from Chapter 2, because it costs an evening of debugging every time: the password only reaches an adapter when the username is also set.

PART II

Filesystem Protocols

Ten adapters that make remote storage look like a disk: files you read, write, and append; directories you list in the format your DOS expects; rename, delete, mkdir, and seek where the backend allows it. They share one machinery, documented first, so each protocol chapter is the differences only.

CHAPTER 7

The Filesystem Model

Every adapter in this part derives from `NetworkProtocolFS`, the layer that turns "a URL" into "a file or a directory." Learn its behavior here; the ten protocol chapters then only describe what each backend adds, omits, or gets subtly wrong.

7.1 Files: open, read, write, seek

A file open (aux1 = 4, 8, 9, or 12) mounts the server, resolves the path, and opens a file handle. Reads pull from the backend into the channel's receive buffer and count `fileSize` down, so status reports the bytes remaining; when both the file and the buffer are empty, error 136. Writes translate the transmit buffer once and hand it to the backend. Seek (`$25`) and tell (`$26`) work wherever the backend can honor them — TNFS, SD, SMB, NFS, SFTP, and (for GET requests) HTTP — and after a seek, `avail` counts from the new position.

One base-class kindness is worth knowing: if a READ open cannot find the exact path, the adapter lists the parent directory and compares **crunched** 8.3 names — so `N:TNFS://server/LONGFILE.TXT` typed from a machine that shortened `LongFilename.txt` still finds its file. The recovered long name replaces the path, and the open proceeds.

7.2 Directories: one listing, many costumes

A directory open (aux1 = 6) walks the entries and renders them into text — and here aux2 is **not** a translation mode but a **format selector**:

aux2	Format	Rendering
< 128	short	classic 8.3 "DOS 2" lines — crunched name, size in sectors (of `FSSectorSize` bytes), a lock flag
128	LONG	long filename padded to the platform width (37 default, 30 on ADAM, 31 on CoCo), size right-justified; directories get a trailing `/`; overlong names wrap to a second line
129	A2COL80	Apple II 80-column long form
130	GDRIVE	long form with the provider's file ID appended (Chapter 16)
131	RAW	the exact filename plus line ending, nothing else — `ls -F` style, directories marked `/`; the format for programs
132	A2CAT	ProDOS 40-column `CAT`: name, type, blocks, modified date, with header and `BLOCKS FREE` trailer
133	A2CATALOG	ProDOS 80-column `CATALOG`: adds created date and file length columns

Table 8. Directory formats, from `DIR_FORMAT` in `FS.h`. Values 128–255 other than those listed fall back to LONG.

Entries whose names begin with `.` or `/` are skipped. The wildcard in the devicespec's final component filters the listing — `*` and `?` match as you expect, and the spellings `*.*`, `**`, and `-` are all normalized to `*`. On Atari builds every format except RAW ends with the traditional `999+FREE SECTORS` line; the ProDOS formats report a cheerfully fake `BLOCKS FREE: 65535`. Dates in the ProDOS formats render as `<NO DATE>` unless the backend supplies real timestamps — among these protocols only TNFS does.

> **NOTE**
> Directory text is composed with the platform's display line ending already applied, and translation is forced off for the listing — which is exactly why aux2 was free to mean "format" here. On non-Atari platforms the listing line ending is CR/LF.

7.3 Rename's comma convention

The rename special (`$20`) carries both names in one devicespec, comma separated, destination second:

```
N:TNFS://server/games/OLDNAME.XEX,NEWNAME.XEX
```

Both names live in the same directory — the destination is re-anchored to the source's path. No comma means error 165. The same one-shot rules from Chapter 6 apply: complete devicespec, no open channel needed.

7.4 What "supported" means

Each adapter advertises which of rename, delete, mkdir, and rmdir it implements, and its chapter's capability table in this book is the truth as implemented — including the places where an operation is **advertised but stubbed**, accepting the command and reporting success while doing nothing. Those spots are called out in red ink here and collected in Appendix E, because a program that trusts a stubbed rename deserves to know.

7.5 Limits that apply to all of them

The status `avail` field caps at 65534 even for a gigabyte file — treat it as "at least this much," and just read until 136. Write-mode opens report `avail` = 0. And the receive buffer is the FujiNet's RAM, not yours: a read of **n** bytes may return fewer with no error, meaning "that's what was buffered; ask again."

CHAPTER 8

TNFS

SCHEME	TNFS	CLASS	NetworkProtocolTNFS
FAMILY	Filesystem (NetworkProtocolFS)	PORT	16384/udp
LOGIN	none — TNFS is an open, anonymous protocol		
DOES	read · write · append · read/write · directory · seek · rename · delete · mkdir · rmdir · lock · unlock — the full set, all real		

TNFS — the Trivial Network File System — is the retro community's native file server: a tiny UDP protocol designed for 8-bit clients, servers on everything from a Raspberry Pi to a Windows box, and public archives already serving thousands of disk images. It is the FujiNet's most complete filesystem citizen: every operation in the model is genuinely implemented, and it is the only protocol here that returns real modified **and** created timestamps, so the Apple II `CATALOG` formats show honest dates.

8.1 Devicespec

```
N:TNFS://host[:port]/path/to/file
```

The host is taken from the URL; the mount is always the server's root, with the path resolved beneath it. Credentials are not used. Traffic is plain UDP datagrams — nothing is encrypted, which on a 2026 network makes TNFS a protocol for your LAN and for public read-only archives, not for secrets.

8.2 Reference

Item	Behavior
Open 4 (READ)	open existing, read-only
Open 8 (WRITE)	create-or-truncate, mode 777 on the server
Open 9 (APPEND)	create-or-append
Open 12 (READWRITE)	create if needed, read and write
Open 6 (DIRECTORY)	server-side wildcard listing
Seek `$25` / Tell `$26`	real `lseek` on the server; `avail` recounts from the new position
Lock `$23` / Unlock `$24`	chmod 444 / chmod 644
Transfers	chunked to the TNFS payload limit (512-byte datagrams) transparently

Table 9. TNFS operation summary.

TNFS result	Code	Reported as
success	1	SUCCESS
file not found	170	FILE_NOT_FOUND
read-only filesystem, access denied	167	ACCESS_DENIED
no space on device	162	NO_SPACE_ON_DEVICE
end of file	136	END_OF_FILE
file exists	151	FILE_EXISTS
mount timeout	138	GENERAL_TIMEOUT
anything else	144	GENERAL

Table 10. TNFS error mapping.

8.3 Examples

Reading a text file, as a transaction trace:

```
OPEN    aux1=4 aux2=0   "N:TNFS://fujinet.online/atari/README"
STATUS                  -> avail=1042 conn=1 err=1
READ    len=127         -> first 127 bytes
...repeat until...
STATUS                  -> avail=0 conn=0 err=136
CLOSE
```

A directory of disk images, long format, then a rename:

```
OPEN    aux1=6 aux2=128 "N:TNFS://fujinet.online/atari/games/*.ATR"
READ    ...             -> one line per image, sizes right-justified
CLOSE
RENAME                  "N:TNFS://myserver/dev/BUILD.XEX,STABLE.XEX"
```

And the complete C program — the shape every filesystem example in this book shares:

```c
/* tnfs-cat: print a file from a TNFS server. */
#include <stdio.h>
#include "fujinet-network.h"

char *url = "n:tnfs://fujinet.online/atari/README";
uint8_t buf[256];

int main(void)
{
    int16_t n;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_READ, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 1;

    while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
    /* n == -136 here: end of file. Any other negative n is an error. */

    network_close(url);
    return 0;
}
```

Listing 1. `tnfs-cat.c` — read a whole file. `network_read` returns bytes read, or the negative of the error code; end of file is `-136`.

CHAPTER 9

SD

SCHEME	SD	CLASS	NetworkProtocolSD
FAMILY	Filesystem (NetworkProtocolFS)	PORT	none — no network at all
LOGIN	none		
DOES	read · write · append · read/write · directory · seek · rename · delete · mkdir · rmdir — lock/unlock report 146		

The SD adapter points the whole filesystem machinery at the microSD card in the FujiNet itself. It exists for symmetry and it earns its keep: the same program that reads a TNFS server can read the local card by swapping one scheme, which makes `SD:` the offline fallback, the scratch space, and the place the SSH chapters keep their keys.

9.1 Devicespec

```
N:SD:/path/to/file
```

No host, no port, no credentials — the path starts right after the scheme. If no card is mounted (or fujinet-pc has no SD directory configured), every operation reports error 200, connection refused: the one place in this book where 200 means "no card."

9.2 Reference

Open modes 4, 8, 9, and 12 map straight onto the card filesystem's read/write/append/read-write; anything else is rejected. Rename, delete, mkdir, and rmdir are real. Lock and unlock are deliberately **not** implemented — FAT has no permission bits worth the pretense — and report 146 instead of pretending. Seek and tell are the C library's own, and end-of-file detection distinguishes a true EOF (136) from a short read with an error. Because nothing here ever blocks on a network, the adapter turns the interrupt machinery off while it works.

Card errno	Code	Reported as
no such file (`ENOENT`)	170	FILE_NOT_FOUND
exists (`EEXIST`)	151	FILE_EXISTS
permission (`EACCES`)	167	ACCESS_DENIED
card full (`ENOSPC`)	162	NO_SPACE_ON_DEVICE
out of memory (`ENOBUFS` / `ENOMEM`)	255	COULD_NOT_ALLOCATE_BUFFERS
no card mounted	200	CONNECTION_REFUSED
anything else	144	GENERAL

Table 11. SD error mapping.

9.3 Examples

```
OPEN    aux1=8 aux2=0   "N:SD:/NOTES/TODAY.TXT"
WRITE   "CALL AUNT VI\x9b"
CLOSE
OPEN    aux1=6 aux2=128 "N:SD:/NOTES/*"
READ    ...             -> TODAY.TXT              13
CLOSE
```

```c
/* sd-log: append a line to a log file on the FujiNet's SD card. */
#include <string.h>
#include "fujinet-network.h"

char *url = "n:sd:/LOGS/VISITS.TXT";
char *line = "one more visitor\n";

int main(void)
{
    if (network_init() != FN_ERR_OK) return 1;
    /* fujinet-lib names READ/WRITE/RW only; append is plain aux1 = 9 */
    if (network_open(url, 9, OPEN_TRANS_LF) != FN_ERR_OK)
        return 1;
    network_write(url, (uint8_t *)line, strlen(line));
    network_close(url);
    return 0;
}
```

Listing 2. `sd-log.c` — append with translation mode 2, so the machine's native line ending becomes LF on the card.

CHAPTER 10

SMB

SCHEME	SMB	CLASS	NetworkProtocolSMB
FAMILY	Filesystem (NetworkProtocolFS)	PORT	445/tcp (library default)
LOGIN	URL `user:pass@`, else the stashed username/password specials		
DOES	read · write · append · read/write · directory · seek · mkdir · rmdir — rename, delete, lock, unlock accepted but do nothing		

SMB is the share on your Windows machine, your NAS, or a Samba server — which makes this the adapter that lets an Atari save directly into the folder your PC is already watching. The FujiNet speaks SMB2 through `libsmb2`, with message signing enabled.

10.1 Devicespec

```
N:SMB://[user:pass@]server/share/path/to/file
```

The first path component is the **share name**; the rest is the path within it. Credentials come from the URL, or failing that from the stashed username/password specials. For a directory listing, a trailing wildcard component is honored.

> **NOTE**
> Because all-caps machines are SMB's main customers here, the adapter applies a friendly mangle: a username or password that contains **no lowercase letters at all** is converted to lowercase before use. `MOLLY` becomes `molly` and logs in; a password that is genuinely all-caps, though, will be mangled too — mixed-case or lowercase secrets pass through untouched. Choose server passwords accordingly.

10.2 Reference

Open modes 4, 8 (create), 9 (append-create), and 12 map to the library's POSIX-style flags; directory opens list the share. Mkdir and rmdir are real. Seek is honored by re-reading from an absolute offset. **Rename, delete, lock, and unlock are not implemented** — the commands are accepted and report success, but nothing happens on the server. Error reporting is SMB's weak spot: every library failure maps to the general error 144, so "wrong password," "no such share," and "cable unplugged" all read the same from the computer. When an SMB open fails, check the server name, the share name, and the credentials in that order.

10.3 Examples

```
USERNAME "molly"          ($FD)
PASSWORD "sekrit"         ($FE)
OPEN    aux1=6 aux2=128  "N:SMB://den-pc/atari/*.XEX"
READ    ...              -> the share's XEX files
CLOSE
OPEN    aux1=4 aux2=0    "N:SMB://den-pc/atari/JUMPMAN.XEX"
...read until 136...
```

```c
/* smb-fetch: copy a file from a Windows share to the SD card. */
#include "fujinet-network.h"

char *src = "n:smb://molly:sekrit@den-pc/atari/JUMPMAN.XEX";
char *dst = "n:sd:/GAMES/JUMPMAN.XEX";
uint8_t buf[512];

int main(void)
{
    int16_t n;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(src, OPEN_MODE_READ,  OPEN_TRANS_NONE) != FN_ERR_OK) return 1;
    if (network_open(dst, OPEN_MODE_WRITE, OPEN_TRANS_NONE) != FN_ERR_OK) return 1;

    while ((n = network_read(src, buf, sizeof(buf))) > 0)
        network_write(dst, buf, n);

    network_close(dst);
    network_close(src);
    return 0;
}
```

Listing 3. `smb-fetch.c` — two channels at once: SMB in, SD out, translation off because a binary is a binary.

CHAPTER 11

NFS

SCHEME	NFS	CLASS	NetworkProtocolNFS
FAMILY	Filesystem (NetworkProtocolFS)	PORT	2049 (a URL port overrides both mount and NFS ports)
LOGIN	username/password specials as numeric UID/GID		
DOES	read · write · append · read/write · directory · seek · mkdir · rmdir — rename, delete, lock, unlock accepted but do nothing		

NFS reaches the exports of a UNIX or Linux server — version 3, spoken by `libnfs`. It fills the same role as SMB for households whose file server says `/etc/exports` instead of "sharing tab," and it interoperates with the newer user-space servers (rclone, go-nfs) as well as the kernel one.

11.1 Devicespec

```
N:NFS://host[:port]/export/path/to/file
```

Authentication is classic NFSv3: numeric UNIX IDs, not names. The stashed username special supplies the **UID** and the password special the **GID**, both as decimal strings — `USER 1000` / `PASS 100` in NOS terms. If a port is given it is used for both the mount and NFS services, which is what user-space servers on a single high port want.

The adapter works out the export boundary for you: it first tries to mount the path's own directory, and if the server refuses (kernel servers export only fixed roots), it falls back to mounting the root export and addressing the path beneath it. A path ending in `/` is a directory to list; otherwise the parent is mounted and the leaf addressed.

11.2 Reference

Open modes are as SMB's. Mkdir and rmdir are real. **Rename, delete, lock, and unlock are stubs** — accepted, reported successful, nothing done — despite the adapter advertising rename and delete internally (Appendix E). Seek follows the SMB shape. And like SMB, error reporting collapses to the general 144 for every library failure.

11.3 Examples

```
USERNAME "1000"           (UID)
PASSWORD "100"            (GID)
OPEN    aux1=4 aux2=2    "N:NFS://deepthought/export/motd"
...read until 136...
CLOSE
```

```c
/* nfs-motd: read a file from an NFS export. */
#include <stdio.h>
#include "fujinet-network.h"

char *url = "n:nfs://deepthought/export/motd";
uint8_t buf[256];

int main(void)
{
    int16_t n;
```

```
    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_READ, OPEN_TRANS_LF) != FN_ERR_OK)
        return 1;
    while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
    network_close(url);
    return 0;
}
```

Listing 4. `nfs-motd.c` — translation mode 2 folds the server's LF endings into the machine's own.

CHAPTER 12

FTP

SCHEME	FTP	CLASS	NetworkProtocolFTP
FAMILY	Filesystem (NetworkProtocolFS)	PORT	21/tcp control
LOGIN	none — always anonymous (`anonymous` / `fujinet@fujinet.online`)		
DOES	read · write · directory — append and read/write report 146; rename, delete, mkdir, rmdir, lock, unlock accepted but do nothing		

FTP is here for the archives that still speak it — and plenty do. The adapter is a deliberately simple client: anonymous login, passive transfers, download, upload, and directory listings. It is a **retrieval** protocol in this firmware, not a file-management one.

12.1 Devicespec

```
N:FTP://host[:port]/path/to/file
```

Credentials in the URL, and the stashed username/password specials, are **ignored**: every session logs in as `anonymous` with the password `fujinet@fujinet.online`, the polite convention for anonymous FTP. There is no FTPS/TLS support — like TNFS, treat it as a public-archive protocol.

12.2 Reference

Mode 4 issues `RETR`, mode 8 issues `STOR`; append (9) and read/write (12) report 146, because FTP's data connection is one-way per transfer. Directory opens use the server listing with the usual wildcard filter, with permissions synthesized (FTP listings don't carry them reliably). **Every filesystem special — rename, delete, mkdir, rmdir, lock, unlock — is a stub that reports success without touching the server** (Appendix E).

FTP reply	Code	Reported as
1xx/2xx/3xx positive replies	1	SUCCESS
226 (closing data connection)	136	END_OF_FILE
421 (service closing)	210	SERVICE_NOT_AVAILABLE
430 (bad credentials)	212	INVALID_USERNAME_OR_PASSWORD
450 / 451 / 452	167	ACCESS_DENIED
550 (not found / no access)	170	FILE_NOT_FOUND
other 4xx / 5xx	144	GENERAL

Table 12. FTP reply mapping — the full table collapses the many "fine" replies into SUCCESS; these are the ones that change program behavior.

12.3 Examples

```
OPEN    aux1=6 aux2=128 "N:FTP://ftp.pigwa.net/stuff/collections/*"
READ    ...             -> the mirror's top directory
CLOSE
OPEN    aux1=4 aux2=0   "N:FTP://ftp.pigwa.net/stuff/collections/holmes cd/Holmes 1/Games/
Jumpman.atr"
```

```
...read 92176 bytes...
CLOSE
```

```c
/* ftp-dir: list a directory on an anonymous FTP server. */
#include <stdio.h>
#include "fujinet-network.h"

char *url = "n:ftp://ftp.gnu.org/gnu/*";
uint8_t buf[256];

int main(void)
{
    int16_t n;

    if (network_init() != FN_ERR_OK) return 1;
    /* aux1 = 6: directory; aux2 = 131: RAW format, one name per line */
    if (network_open(url, 6, 131) != FN_ERR_OK)
        return 1;
    while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
    network_close(url);
    return 0;
}
```

Listing 5. `ftp-dir.c` — a directory open is just an open with aux1 = 6; here aux2 picks the RAW format, the one meant for parsing.

CHAPTER 13

SFTP

SCHEME	SFTP	CLASS	NetworkProtocolSFTP
FAMILY	Filesystem (NetworkProtocolFS)	PORT	22/tcp
LOGIN	URL `user:pass@` for password auth; `user@` alone for key auth		
DOES	read · write · append · read/write · directory · seek · rename · delete · mkdir · rmdir · lock · unlock — the full set, all real		

SFTP is the encrypted counterpart to TNFS's innocence: the file side of SSH, reaching any UNIX box, NAS, or hosting account with an SSH daemon — over a connection that is actually private. Alongside TNFS it is the only filesystem adapter with the **complete** operation set genuinely implemented.

13.1 Devicespec

```
N:SFTP://user:pass@host[:port]/path/to/file
```

The URL's shape selects the authentication method, the same convention as the SSH chapter:

- `user:pass@host` — password authentication.
- `user@host` — **public-key** authentication, using the FujiNet's own ed25519 key at `/.ssh/id_ed25519` on the SD card. Chapter 23 generates that key; Chapter 24 installs it on the server. Once those two have run, your devicespecs never carry a password again.

A missing username fails immediately with 212. Paths are absolute on the server unless the account's server chroots them.

> **IMPORTANT**
> The adapter does not verify the server's host key against a known-hosts list — the fingerprint is logged, nothing more. The wire is encrypted, but an active man-in-the-middle would not be detected. Treat SFTP from the FujiNet as safe against eavesdroppers, not against an adversary who owns your network.

13.2 Reference

Open 4 reads; 8 creates-or-truncates; 9 creates-or-appends; 12 opens read/write, creating if needed — new files arrive mode 644. Directory opens list with the usual wildcard. Rename, delete, mkdir (mode 755), rmdir, lock (chmod 444), and unlock (chmod 644) all do exactly what they say, and seek is the server's own 64-bit seek. On a read past the end, the buffer is zero-padded to the requested length and 136 raised — the documented contract for reads, honored here literally.

SFTP status	Code	Reported as
OK	1	SUCCESS
EOF	136	END_OF_FILE
no such file / no such path	170	FILE_NOT_FOUND
permission denied / write protect	167	ACCESS_DENIED
file already exists	151	FILE_EXISTS
auth failure	167	ACCESS_DENIED
missing username / empty password	212	INVALID_USERNAME_OR_PASSWORD

SFTP status	Code	Reported as
anything else	144	GENERAL

Table 13. SFTP error mapping.

13.3 Examples

```
OPEN    aux1=6 aux2=128 "N:SFTP://fuji@deepthought/home/fuji/atari/*"
READ    ...             -> the directory, over an encrypted channel
CLOSE
RENAME                  "N:SFTP://fuji@deepthought/home/fuji/atari/A.XEX,B.XEX"
```

```c
/* sftp-put: upload a file to an SSH server using the FujiNet's key. */
#include <string.h>
#include "fujinet-network.h"

char *url = "n:sftp://fuji@deepthought/home/fuji/hello.txt";
char *body = "sent from an 8-bit machine, encrypted end to end\n";

int main(void)
{
    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_WRITE, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 1;
    network_write(url, (uint8_t *)body, strlen(body));
    network_close(url);
    return 0;
}
```

Listing 6. `sftp-put.c` — no password in the program at all: the URL's password-less shape selects key authentication.

CHAPTER 14

HTTP and HTTPS

SCHEME	**HTTP • HTTPS**	**CLASS**	`NetworkProtocolHTTP`
FAMILY	Filesystem (NetworkProtocolFS) — plus WebDAV	**PORT**	`80 / 443`
LOGIN	URL `user:pass@` (HTTP auth); headers settable for tokens		
DOES	GET · PUT · POST · DELETE · WebDAV directory · seek (GET) · header get/set · rename (MOVE) · delete · mkdir (MKCOL)		

HTTP is the biggest chapter in this book because it is the biggest protocol in the world: the adapter is at once a file fetcher, a REST client with full header control, a form poster, and a WebDAV filesystem. Every JSON example in Part I rides on it. It repays careful reading — mostly because of one design decision, explained first.

14.1 The lazy transaction

Opening `N:HTTPS://…` does **not** send the request. The open parses the URL, creates the client, and returns — the request actually fires at the **first status call** on the channel. The reason is headers: between open and that first status you can still add request headers and staged POST data, things HTTP requires **before** the request goes out. The firmware raises an interrupt after open precisely to invite that first status.

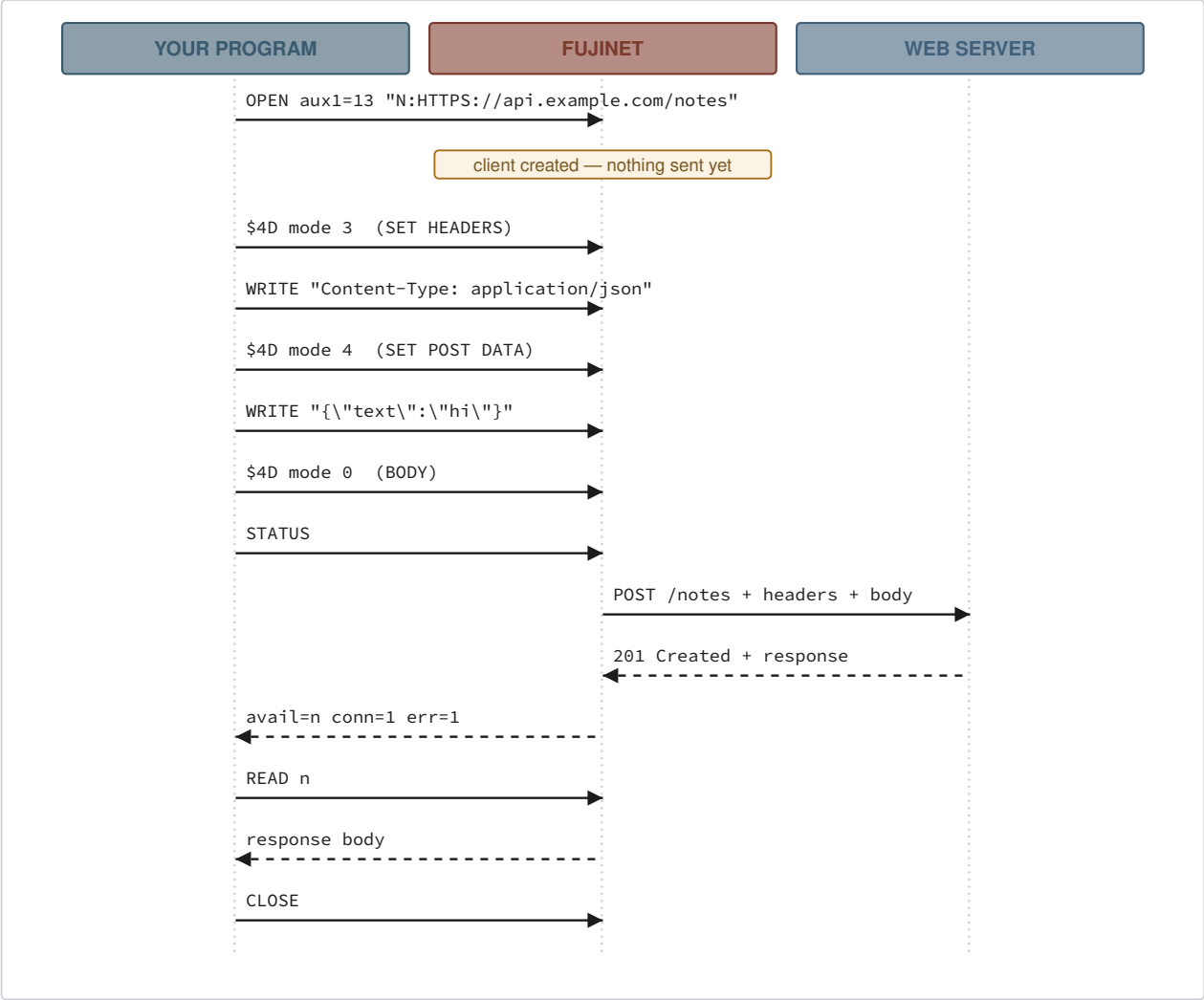

Figure 3. A POST with a header, in full. The request leaves on the first STATUS — everything before it is staging.

Two corollaries. A plain GET works exactly as you expect — open, status, read — because the status you were going to issue anyway is what pulls the trigger. And an HTTP error (404, 500…) surfaces in that first status's error byte, mapped by the table at the end of this chapter.

14.2 Access modes are methods

aux1	Method	Behavior
4	GET	fetch; the path gets filename resolution and URL encoding
12	GET	"pure" GET — path sent verbatim, headers settable
8	PUT	write the body; sent when the channel closes
14	POST	PUT-with-headers in name, transmitted as POST
13	POST	write staged data, read the response
5	DELETE	issue DELETE, no header staging
9	DELETE	DELETE with header staging
6	—	WebDAV PROPFIND directory listing

Table 14. aux1 as HTTP method. Modes 4 and 8 treat the URL like a **file** path (encoding spaces, resolving names); every other mode sends your path byte-for-byte — use 12/13/14 for APIs whose URLs must not be touched.

14.3 The HTTP channel modes

The `$4D` special (HTTP channel mode; aux2 carries the mode) points the channel's reads and writes at different parts of the transaction:

aux2	Mode	READ returns	WRITE means
0	BODY	the response body	stage PUT body data
1	COLLECT HEADERS	— (131)	name a response header to capture (GET modes only)
2	GET HEADERS	captured header values, one per read	— (135)
3	SET HEADERS	— (131)	add a request header, `Name: value`
4	SET POST DATA	— (131)	stage POST/PUT body data

Table 15. HTTP channel modes. Wrong-direction operations fail with 131/135; modes that need staging (1, 3) only work before the transaction fires.

The header dance for reading a response header is: before the first status, switch to mode 1 and write each header **name** you care about; after the transaction, switch to mode 2 and read the values back in the order you named them; switch to mode 0 for the body. Header collection is armed only in the header-capable modes (5, 9, 12, 13, 14).

14.4 WebDAV: the web as a disk

A directory open sends a depth-1 PROPFIND and renders the collection like any other directory — name, size, directory flag — so a WebDAV share (Nextcloud, Apache mod_dav, a NAS) works exactly like TNFS from BASIC's point of view. If the server answers 405 or 408 — "PROPFIND? never heard of it" — the adapter quietly retries as a plain GET, so opening a **normal** web page with aux1 = 6 yields its raw HTML instead of an error. The filesystem specials complete the picture: rename becomes WebDAV `MOVE`, delete becomes `DELETE`, mkdir becomes `MKCOL`, and rmdir is an alias for delete.

14.5 Seek: HTTP Range

For a GET channel in body mode, seek (`$25`) is real: the adapter re-issues the request with a `Range: bytes=n-` header, reusing the connection. A server that answers 206 gives you a true random-access file over the web; a server that ignores Range and answers 200 is handled by skipping bytes internally (correct, but the skipped bytes cross the network). Tell (`$26`) reports the position either way; seeking on a non-GET channel returns the invalid-point error 166.

14.6 Reference

HTTP status	Code	Reported as
200–208, 226	1	SUCCESS
401, 402, 403, 407	212	INVALID_USERNAME_OR_PASSWORD
404, 410	170	FILE_NOT_FOUND
405	146	NOT_IMPLEMENTED
408	138	GENERAL_TIMEOUT
423, 451	167	ACCESS_DENIED
other 4xx	214	CLIENT_GENERAL
5xx	215	SERVER_GENERAL
connect failure (int. 901)	207	NOT_CONNECTED
anything else	144	GENERAL

Table 16. HTTP status mapping.

Constants worth knowing: staged POST data is unbounded (RAM is the limit); response headers read back in `$FC` -style GET HEADERS mode are terminated with `$9B` ; and HTTPS certificate trust comes from the firmware's bundled CA store.

14.7 Examples

The whole-web-page fetch:

```
OPEN    aux1=4 aux2=0   "N:HTTPS://www.gnu.org/licenses/gpl-3.0.txt"
STATUS                  -> avail=..., err=1     (request fired here)
READ    ...until 136
CLOSE
```

A REST GET with a bearer token and a captured response header:

```
OPEN    aux1=12 aux2=0  "N:HTTPS://api.example.com/v1/me"
$4D mode 1 ; WRITE "Content-Type"          (collect this response header)
$4D mode 3 ; WRITE "Authorization: Bearer MYTOKEN"
$4D mode 0 ; STATUS                        (fires the GET)
READ    ...body...
$4D mode 2 ; READ                          -> "application/json"
CLOSE
```

```c
/* http-post: send JSON to an API and print the response.
 * fujinet-lib wraps the whole header/staging dance in one call. */
#include <stdio.h>
#include <string.h>
#include "fujinet-network.h"

char *url = "n:https://httpbin.org/post";
uint8_t buf[512];

int main(void)
{
    int16_t n;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_HTTP_POST, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 1;

    network_http_start_add_headers(url);
    network_http_add_header(url, "Content-Type: application/json");
    network_http_end_add_headers(url);

    if (network_http_post(url, "{\"who\":\"fujinet\"}") != FN_ERR_OK)
        return 1;

    while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
    network_close(url);
    return 0;
}
```

Listing 7. `http-post.c` — the C spelling of the sequence diagram above.

CHAPTER 15

S3

SCHEME	S3	CLASS	NetworkProtocolS3
FAMILY	Filesystem (NetworkProtocolFS)	PORT	443 (or 80 with `?tls=0`)
LOGIN	URL `ACCESSKEY:SECRETKEY@`, else keys stored in FujiNet config		
DOES	read · write · append · read/write · directory · rename (copy+delete) · delete · mkdir · rmdir — no lock/unlock, no seek		

S3 puts an object store behind the `N:` device — real Amazon S3, or any compatible service: MinIO on your LAN, Wasabi, a Ceph gateway. The adapter signs every request with AWS Signature V4, correctly and entirely inside the FujiNet, which means a 6502 can PUT an object into a bucket with 2026-grade request signing and never know it.

15.1 Devicespec

```
S3://[ACCESSKEY:SECRETKEY@]endpoint[:port]/bucket/key?region=..&tls=0|1
```

The first path component is the bucket; the rest is the object key. Requests use path-style addressing, so custom endpoints work without wildcard DNS. Three things resolve with fallbacks:

- **Credentials**: URL userinfo first, else the access/secret keys stored in the FujiNet's configuration (set once in the web UI).
- **Region**: `?region=` first, else the configured region, else `us-east-1` (what MinIO expects).
- **TLS**: `?tls=0` (or `false` / `no`) for plain HTTP, `?tls=1` to force it, else the configured default. A redundant `:443` / `:80` port is trimmed so the signature's Host header matches.

An empty endpoint or bucket fails with 165 before anything is sent.

15.2 Reference

Reads stream a GET. Writes buffer in the FujiNet and go out as **a single PUT when the channel closes** — up to 512 KB; beyond that the write fails with 162, and nothing partial is ever sent. Append downloads the existing object first, then behaves like write. Directory opens use `ListObjectsV2` with `/` as delimiter, paginated transparently, wildcard filtering applied to objects (prefixes — "folders" — always list). Mkdir creates a zero-byte folder-marker object; rmdir deletes one; rename is the S3 idiom of copy-then-delete; lock, unlock, and seek do not exist here.

S3 response	Code	Reported as
200 / 201 / 204 / 206	1	SUCCESS
403	167	ACCESS_DENIED — bad signature, clock, or policy
404	170	FILE_NOT_FOUND
409	151	FILE_EXISTS
no response at all	207	NOT_CONNECTED
anything else	144	GENERAL

Table 17. S3 error mapping.

15.3 Examples

```
OPEN    aux1=6 aux2=128 "S3://minio.local:9000/retro/*?tls=0"
READ    ...             -> the bucket's objects
CLOSE
OPEN    aux1=8 aux2=0   "S3://minio.local:9000/retro/saves/game1.sav?tls=0"
WRITE   ...save data...
CLOSE                   (the PUT happens here)
```

```c
/* s3-save: store a high-score file as an S3 object. */
#include <string.h>
#include "fujinet-network.h"

/* keys stored in FujiNet config; none in the program */
char *url = "n:s3://s3.amazonaws.com/my-retro-bucket/scores/today.txt"
            "?region=us-east-2";
char *scores = "MOLLY 12400\nDUKE 9950\n";

int main(void)
{
    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_WRITE, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 1;
    network_write(url, (uint8_t *)scores, strlen(scores));
    /* the object uploads as one signed PUT at close */
    return network_close(url) == FN_ERR_OK ? 0 : 1;
}
```

Listing 8. `s3-save.c` — note that success is only known at `network_close`, when the PUT actually happens.

CHAPTER 16

GDRIVE

SCHEME	**GDRIVE**	**CLASS**	NetworkProtocolGDRIVE
FAMILY	Filesystem (NetworkProtocolFS)	**PORT**	443 (Google APIs)
LOGIN	OAuth — authorized once in the FujiNet web UI; nothing on the bus		
DOES	read · write · append · read/write · directory · delete · mkdir · rmdir — no rename, no lock/unlock, no seek		

GDRIVE mounts your Google Drive. The OAuth handshake happens once, in the FujiNet's web configuration page; after that the firmware silently refreshes its access token through the FujiNet auth relay (so Google's client secret never lives on your device) and your programs just open paths. It shares that one Google authorization with GMAIL and GCAL — one consent covers all three.

16.1 Devicespec

```
GDRIVE:///path/to/file.txt
```

The host portion is empty — there is only one Drive per authorization. The path looks like a filesystem path, but Drive is really an object graph, so the adapter walks it component by component, resolving each name to a file ID. Consequences worth knowing: name lookups are case-sensitive exact matches; **shortcuts are followed** (a shortcut to a folder lists like the folder); items in the trash are invisible; and if two files share a name in one folder — legal in Drive — you get whichever Google returns first.

16.2 Reference

Reads stream the file content. Writes buffer up to 64 KB and upload as a single multipart request when the channel closes — new file or update, as appropriate; beyond the limit the write fails (144) and nothing is sent. Append pre-downloads the existing content. Directory opens list a folder (up to 1000 entries, name order); opening a **file** path as a directory yields a one-entry listing of that file, handy for an existence check. The GDRIVE directory format (aux2 = 130) appends each entry's Drive file ID — the hook that lets a program address files by ID later. Delete is real; mkdir creates a folder; rmdir is an alias for delete (it will happily remove a non-empty folder — Drive semantics); **there is no rename**.

Failures largely report specific codes set at the failure site: 170 for an unresolvable path, 207 when the token refresh fails ("mount"), 136 at end of content, 144 for API errors. If the FujiNet has never been authorized (or the refresh token was revoked), every open fails at mount — re-authorize in the web UI.

16.3 Examples

```
OPEN    aux1=6 aux2=130 "GDRIVE:///Atari/Docs/*"
READ    ...             -> entries with Drive IDs appended
CLOSE
OPEN    aux1=8 aux2=0   "GDRIVE:///Atari/Docs/notes.txt"
WRITE   ...text...
CLOSE                   (multipart upload happens here)
```

```c
/* gdrive-note: append today's note to a file in Google Drive. */
#include <string.h>
```

```c
#include "fujinet-network.h"

char *url = "n:gdrive:///Atari/journal.txt";
char *entry = "Tuesday: got S3 working from BASIC.\n";

int main(void)
{
    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, 9 /* append */, OPEN_TRANS_LF) != FN_ERR_OK)
        return 1;
    network_write(url, (uint8_t *)entry, strlen(entry));
    return network_close(url) == FN_ERR_OK ? 0 : 1;
}
```

Listing 9. `gdrive-note.c` — append re-uploads the whole file at close; fine for notes, wrong for gigabytes.

CHAPTER 17

ONEDRIVE

SCHEME	**ONEDRIVE**	**CLASS**	NetworkProtocolONEDRIVE
FAMILY	Filesystem (NetworkProtocolFS)	**PORT**	443 (Microsoft Graph)
LOGIN	OAuth — authorized once in the FujiNet web UI; nothing on the bus		
DOES	read · write · append · read/write · directory · rename · delete · mkdir · rmdir — no lock/unlock, no seek		

ONEDRIVE is GDRIVE's sibling for the Microsoft cloud, and the two chapters rhyme on purpose: same empty host, same buffered-upload model, same one-time web-UI authorization (against your Microsoft account, via the same FujiNet auth relay). The differences come from Microsoft Graph being path-native — and they are mostly in OneDrive's favor.

17.1 Devicespec

```
ONEDRIVE:///path/to/file.txt
```

Graph addresses items by path directly, so there is no ID walk: opens are a single request deep no matter how nested the path, name collisions cannot happen, and — the practical win — **rename is implemented**, as a single metadata patch. The comma convention from Chapter 7 applies as usual.

17.2 Reference

Reads stream content; writes buffer up to 64 KB and upload as one simple PUT at close; append pre-downloads. Directory opens list a folder's children (up to 1000, name order), and a file path again yields a one-entry listing. Delete, mkdir (failing with 151 if the name exists), and rmdir (alias for delete) are real, plus the rename GDRIVE lacks. There is no ID-flavored directory format here — aux2 = 130 renders as plain LONG. Error behavior matches GDRIVE's pattern: specific codes from the failure site, 144 for the remainder, and a mount failure means "re-authorize in the web UI."

17.3 Examples

```
RENAME                  "ONEDRIVE:///Retro/draft.txt,final.txt"
OPEN    aux1=4 aux2=0   "ONEDRIVE:///Retro/final.txt"
...read until 136...
CLOSE
```

```c
/* onedrive-fetch: read a OneDrive file, renaming it first. */
#include <stdio.h>
#include "fujinet-network.h"

uint8_t buf[256];

int main(void)
{
    int16_t n;
    char *url = "n:onedrive:///Retro/final.txt";

    if (network_init() != FN_ERR_OK) return 1;
```

```
        network_fs_rename("n:onedrive:///Retro/draft.txt,final.txt");

        if (network_open(url, OPEN_MODE_READ, OPEN_TRANS_NONE) != FN_ERR_OK)
            return 1;
        while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
            buf[n] = 0;
            printf("%s", buf);
        }
        network_close(url);
        return 0;
    }
```

Listing 10. `onedrive-fetch.c` — `network_fs_rename` is the one-shot special from Chapter 6: no open channel involved.

PART III

Stream Protocols

Five adapters that move bytes rather than files: raw TCP in both directions — including a listening server your Atari can answer — UDP datagrams, negotiated Telnet, a real SSH shell, and WebSockets plain and secure. The four-beat lifecycle is all there is; these chapters are about connection shapes and edge behavior.

CHAPTER 18

TCP

SCHEME	TCP	CLASS	NetworkProtocolTCP
FAMILY	Stream (NetworkProtocol)	PORT	23 when omitted (client); yours to choose (server)
LOGIN	none		
DOES	connect · listen/accept · read · write · close client — plus every channel-mode trick from Part I, since JSON can ride any stream		

TCP is the bare metal of this part: a socket, nothing more. It is the adapter for BBSes and MUDs, for talking to your own server programs, for any protocol this book **doesn't** have a chapter for — and, unusually for a retro peripheral, it listens as well as calls. A FujiNet machine can be the server.

18.1 Devicespec — four shapes

Devicespec	Meaning
`N:TCP://host:port/`	connect to `host:port`
`N:TCP://host/`	connect to `host:23` — the Telnet-age default
`N:TCP://:port/`	**listen** on `port`; the channel becomes a server
`N:TCP://`	an empty channel — no socket until you need one

Table 18. The host decides the direction: present means client, absent means server.

Open modes are conventionally 12 (read/write); the mode is accepted but the socket is always bidirectional. Translation applies as usual — mode 0 for binary protocols, a line mode for chatting with text services.

18.2 Server mode

After a listening open, status reports `connected = 1` when a client is **waiting**. Accept it with the `$41` special; from then on reads and writes address that client, and status reflects the client's connection. Drop the client with `$63` and the socket listens again. One client at a time — this is a 1980s machine's server, not nginx.

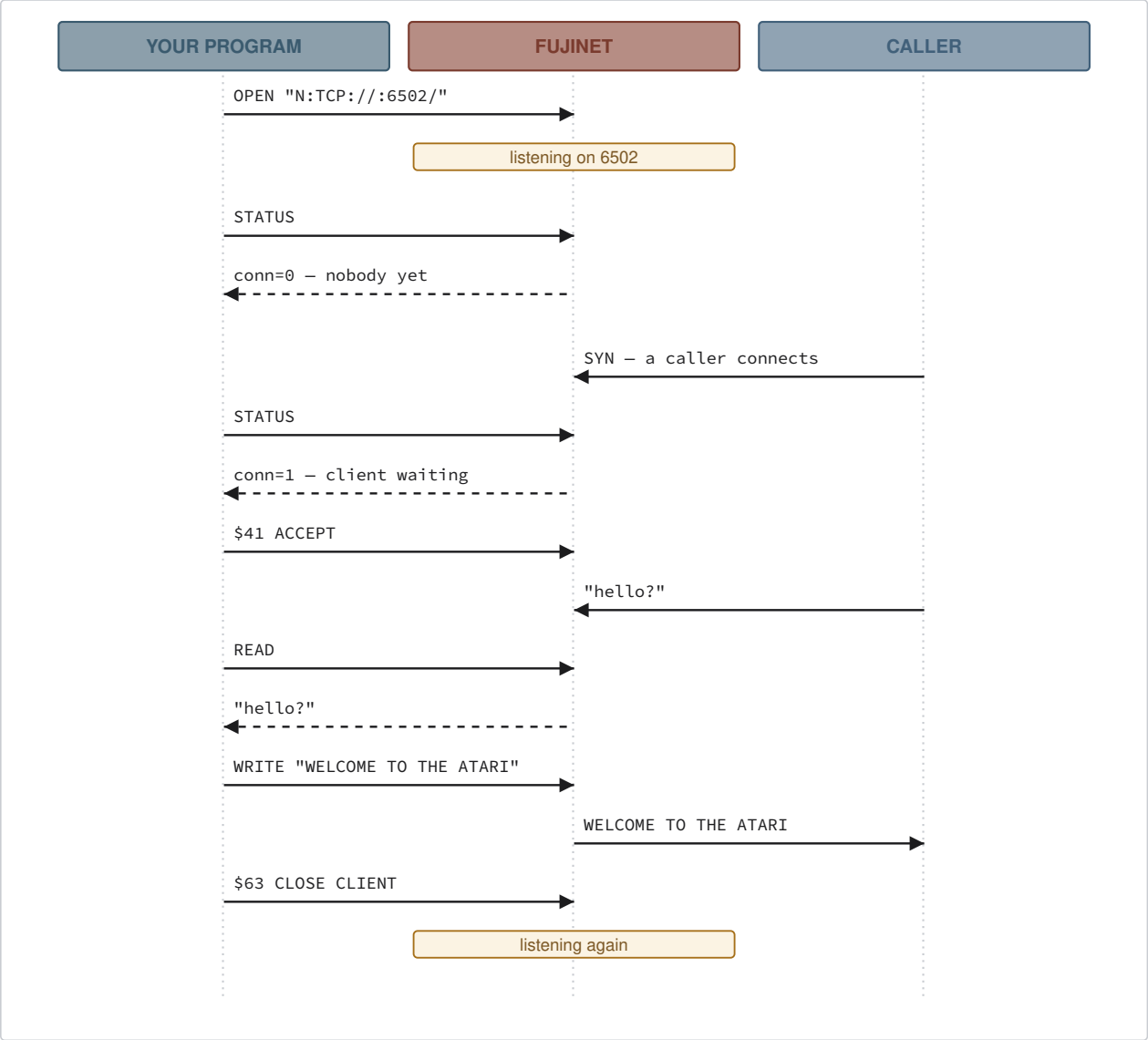

Figure 4. A TCP listener's life. Status is the doorbell; accept opens the door; close-client hangs up and keeps listening.

18.3 Reference

Condition	Code	Notes
connect refused / failed	200	CONNECTION_REFUSED
short read or write	202	SOCKET_TIMEOUT — the peer stalled
peer reset	204	CONNECTION_RESET
write with no client	207	NOT_CONNECTED
accept with no server socket	208	SERVER_NOT_RUNNING
accept with nobody waiting	209	NO_CONNECTION_WAITING
accepted client already gone	204	CONNECTION_RESET
peer closed (status, client mode)	136	END_OF_FILE

Table 19. TCP conditions. The socket errno map from Chapter 5 fills in the rest.

One rough edge to know: the close-client special **reports an error even when it succeeds** — the disconnect happens, but the bus transaction returns the error state. Programs should ignore the status of `$63` itself and trust the next status call (Appendix E). And a firmware subtlety that works in your favor: TCP is the one protocol the interrupt-time poller queries for real, so the doorbell in the diagram above rings promptly.

18.4 Examples

```c
/* tcp-quote: fetch a "quote of the day" (RFC 865) — the original
 * one-transaction TCP service. */
#include <stdio.h>
#include "fujinet-network.h"

char *url = "n:tcp://djxmmx.net:17/";
uint8_t buf[256];

int main(void)
{
    int16_t n;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_RW, OPEN_TRANS_LF) != FN_ERR_OK)
        return 1;
    while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
    network_close(url);
    return 0;
}
```

Listing 11. `tcp-quote.c` — a QOTD server talks first, so this is open-read-close; the read loop ends when the server hangs up (`-136`).

For a full interactive netcat — keyboard to socket, socket to screen — see the closing chapter of every programmer's guide in Appendix F; it is the traditional finale, and Part VI shows its skeleton in each language.

CHAPTER 19

UDP

SCHEME	**UDP**	**CLASS**	`NetworkProtocolUDP`
FAMILY	Stream (NetworkProtocol)	**PORT**	`none — the port is required`
LOGIN	none		
DOES	send · receive · set destination (`$44`) · get remote (`$72` , fujinet-pc only) — connectionless; status always reports connected		

UDP is datagrams: no connection, no ordering, no guarantee — and no equal for LAN-party games, service discovery, and telemetry, where "newest packet wins" beats "reliable but late." The adapter is equally happy as sender, receiver, or both.

19.1 Devicespec

```
N:UDP://host:port/    (talk to host)      N:UDP://:port/    (listen on port)
```

The port is mandatory — there is no default, and an open without one fails. With a host, writes go to that host; without one, the channel listens. On the ESP32 firmware the named port is also the local bind port in both forms; fujinet-pc binds an ephemeral local port when a destination host is given, which matters if the peer replies to the source port rather than a known one.

Two specials complete the story:

- **Set destination** (`$44`) retargets writes mid-session; the payload is `N:host:port` . This is how one channel polls several peers.
- **Get remote** (`$72`) reports `ip:port` of the last sender — fujinet-pc builds only; the ESP32 firmware does not implement it.

And one behavior does the same job implicitly: when a datagram arrives, **the channel's destination auto-learns the sender**, so a listener that just writes after reading answers whoever last spoke. Simple request/reply servers need no address bookkeeping at all.

19.2 Reference

Reads return one datagram's payload (or as much as you asked for); status `avail` reports what is queued, `connected` is always 1 — connectionless protocols don't disconnect. A write with no destination set (never given, never learned) fails 207. Delivery failures are UDP-invisible: a datagram into the void reports success, because that is what UDP means. Multicast address detection exists in the adapter but is not wired to anything yet — sending **to** a multicast group works like any send; joining groups for receive is not implemented.

19.3 Examples

```
OPEN    aux1=12 aux2=0  "N:UDP://:5004/"        (listen)
STATUS                  -> avail=0 conn=1
...peer sends "PING"...
STATUS                  -> avail=4
READ 4                  -> "PING"               (destination learned)
WRITE   "PONG"                                  (goes back to sender)
```

```c
/* udp-beacon: shout our presence, listen for an answer. */
#include <stdio.h>
#include <string.h>
#include "fujinet-network.h"

char *url = "n:udp://192.168.1.255:5004/";
uint8_t buf[64];

int main(void)
{
    int16_t n;
    uint16_t bw; uint8_t conn, err;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_RW, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 1;

    network_write(url, (uint8_t *)"ANYBODY?", 8);

    for (;;) {                      /* poll until someone answers */
        network_status(url, &bw, &conn, &err);
        if (bw > 0) break;
    }
    n = network_read(url, buf, sizeof(buf) - 1);
    if (n > 0) { buf[n] = 0; printf("heard: %s\n", buf); }

    network_close(url);
    return 0;
}
```

Listing 12. `udp-beacon.c` — a broadcast question; the auto-learned destination means a follow-up write would reach whoever answered.

CHAPTER 20

TELNET

SCHEME	**TELNET**	**CLASS**	`NetworkProtocolTELNET`
FAMILY	Stream (subclass of TCP)	**PORT**	23
LOGIN	none (the service prompts, you type)		
DOES	everything TCP client mode does, with Telnet option negotiation handled invisibly — terminal type and window size announced for you		

Raw TCP works for many BBSes — until one sends Telnet option negotiations and your screen fills with stray `ÿû` bytes. The TELNET adapter is TCP plus a real Telnet state machine: negotiations are answered invisibly, and your program sees only clean text. Use it for anything that calls itself a telnet service; fall back to TCP only for servers that dislike negotiation.

20.1 Devicespec

```
N:TELNET://host[:port]/?term=ansi&cols=40&rows=24
```

All three query parameters are optional. `term` names the terminal type offered when the server asks (default `dumb` — honest for most 8-bit screens; claim `ansi` or `vt100` only if your terminal program renders the escapes). `cols` / `rows` advertise the window size — and only if you give **both** does the adapter mention window sizing to the server at all, so old services see byte-for-byte classic behavior unless you opt in.

Behind the scenes the adapter refuses to echo, offers terminal-type, and declines compression and MUD server status — the negotiation set a terminal client should have. Server mode is not part of this adapter: listen with TCP if you want callers.

20.2 Examples

```
OPEN    aux1=12 aux2=3  "N:TELNET://bbs.fozztexx.com/?term=ansi&cols=40&rows=24"
STATUS / READ / WRITE   ...you are on the BBS; negotiation never reaches you...
CLOSE
```

```c
/* telnet-banner: connect to a BBS and show its first screen. */
#include <stdio.h>
#include "fujinet-network.h"

char *url = "n:telnet://bbs.fozztexx.com/?term=dumb";
uint8_t buf[256];

int main(void)
{
    int16_t n, total = 0;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_RW, OPEN_TRANS_CRLF) != FN_ERR_OK)
        return 1;
    while (total < 2000 && (n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
```

```
            printf("%s", buf);
            total += n;
        }
        network_close(url);
        return 0;
    }
```

Listing 13. `telnet-banner.c` — the negotiation bytes a raw TCP connection would show as garbage simply never appear.

CHAPTER 21

SSH

SCHEME	SSH	CLASS	NetworkProtocolSSH
FAMILY	Stream (NetworkProtocol)	PORT	22
LOGIN	URL `user:pass@` for password auth; `user@` alone for key auth		
DOES	an interactive login shell over an encrypted channel, with a real PTY — terminal type and size settable in the query string		

SSH gives your vintage machine a genuine, encrypted login session on a modern computer: a shell with a PTY, running over the same key or password conventions as SFTP (Chapter 13). Every terminal program that can drive a TCP channel can drive this one — the encryption is the FujiNet's problem.

21.1 Devicespec

```
N:SSH://user:pass@host[:port]/?term=vt100&cols=80&rows=24
```

The URL's shape selects authentication exactly as in SFTP: password present means password auth; absent means the ed25519 key from the SD card (`/.ssh/id_ed25519` — Chapters 23 and 24 create and install it). A missing username is error 212 immediately. The query parameters set the PTY: `term` defaults to `vanilla`, `cols` to 80, `rows` to 24 — set `cols=40` (or 32) so full-screen programs on the server wrap for your real screen.

IMPORTANT
As with SFTP, the server's host key is logged, not verified. And in password auth the password rides inside the devicespec — visible to anything that can read your program. Prefer key auth for anything that matters; it is two chapters away.

21.2 Reference

After authentication the adapter requests a PTY of the given size, opens a shell, and goes non-blocking: reads drain what the shell has produced (up to a 64 KB internal buffer), writes feed keystrokes to it, status reports the channel alive until the remote shell exits, then 136. Failure codes: 207 for session or TCP-level failures, 167 when the server rejects the credentials, 212 for missing ones, 144 for the various library-level failures (including a server that does not offer the auth method the URL's shape selected).

21.3 Examples

```
OPEN    aux1=12 aux2=2  "N:SSH://fuji@deepthought/?term=dumb&cols=40"
READ    ...             -> "Linux deepthought 6.9 ... fuji@deepthought:~$ "
WRITE   "uptime\x9b"    (mode 2: the $9B becomes LF on the wire)
READ    ...             -> " 15:02:11 up 42 days ..."
WRITE   "exit\x9b"
STATUS                  -> err=136 when the shell ends
CLOSE
```

```c
/* ssh-uptime: log in with the FujiNet's key, run one command. */
#include <stdio.h>
#include <string.h>
```

```c
#include "fujinet-network.h"

char *url = "n:ssh://fuji@deepthought/?term=dumb";
uint8_t buf[512];

int main(void)
{
    int16_t n;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_RW, OPEN_TRANS_LF) != FN_ERR_OK)
        return 1;

    network_write(url, (uint8_t *)"uptime; exit\n", 13);
    while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
    network_close(url);
    return 0;
}
```

Listing 14. `ssh-uptime.c` — a one-command session; the read loop ends when the remote shell exits.

CHAPTER 22

WS and WSS

SCHEME	WS • WSS	CLASS	NetworkProtocolWS / NetworkProtocolWSS
FAMILY	Stream (NetworkProtocol; WSS = WS over TLS)	PORT	80 / 443
LOGIN	URL `user:password@` passed through in the handshake URL		
DOES	WebSocket client — one write is one frame, reads drain incoming frames; `?frame=text` selects text frames, default binary		

WebSocket is how the modern web does live data — chat backends, multiplayer game lobbies, ticker feeds — and these adapters make your machine a first-class client. `WS` speaks plain; `WSS` is byte-for-byte the same adapter with TLS switched on. Both hide the HTTP upgrade handshake and all frame bookkeeping.

22.1 Devicespec

```
N:WSS://host[:port]/path?frame=text
```

The URL is rebuilt lowercase and handed to the WebSocket client; an empty host fails with 165. The one option, `?frame=text` (or `txt`), sends outgoing frames as TEXT — some servers (and most JSON APIs over WebSocket) require it; everything else, including the default, sends BINARY. Incoming frames of either kind read the same.

22.2 Reference

The connect handshake gets 10 seconds before failing with 200. After that: **each write becomes exactly one WebSocket frame** — build your message completely, then write it once, because a server will parse each frame as a message. Reads drain whatever frames have arrived; status reports bytes waiting and connection state, turning to 136 when the server closes. Mid-stream failures use the familiar codes: 202 for a stalled or partial transfer, 207 when writing after the connection dropped. WSS certificate trust follows the firmware's CA store, like HTTPS.

22.3 Examples

```
OPEN    aux1=12 aux2=0  "N:WSS://ws.postman-echo.com/raw?frame=text"
WRITE   "HELLO FROM 1979"       (one frame)
STATUS                  -> avail=15
READ 15                 -> "HELLO FROM 1979"    (the echo)
CLOSE
```

```c
/* ws-echo: round-trip one message through a WebSocket echo server. */
#include <stdio.h>
#include <string.h>
#include "fujinet-network.h"

char *url = "n:wss://ws.postman-echo.com/raw?frame=text";
uint8_t buf[128];

int main(void)
{
```

```
    int16_t n;
    uint16_t bw; uint8_t conn, err;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_RW, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 1;

    network_write(url, (uint8_t *)"HELLO FROM 1979", 15);

    do network_status(url, &bw, &conn, &err); while (bw == 0 && conn);
    n = network_read(url, buf, sizeof(buf) - 1);
    if (n > 0) { buf[n] = 0; printf("echo: %s\n", buf); }

    network_close(url);
    return 0;
}
```

Listing 15. `ws-echo.c` — one frame out, one frame back, over TLS the whole way.

PART IV

Session Utilities

Five adapters that are not quite network protocols and all the more useful for it: a key generator and a key installer that turn the SSH family passwordless, a clipboard shared with the web UI, a whole CP/M machine behind a devicespec, and the test skeleton every new adapter is born from.

CHAPTER 23

SSH.KEYGEN

SCHEME	**SSH.KEYGEN**	CLASS	NetworkProtocolSSHKeygen
FAMILY	Utility (NetworkProtocol)	PORT	none — everything happens on the FujiNet
LOGIN	none		
DOES	generate the FujiNet's ed25519 key pair onto the SD card; open performs the work, read returns the report		

The SSH and SFTP chapters kept promising a key; this adapter mints it. One open generates an ed25519 key pair into `/.ssh/` on the SD card — private key in OpenSSH format, public key as a ready-to-paste `authorized_keys` line commented `FujiNet`. Run it once per SD card; Chapter 24 does the installing.

23.1 Devicespec

```
N:SSH.KEYGEN://ed25519/          N:SSH.KEYGEN://ed25519/?overwrite=1
```

The host field names the algorithm, and `ed25519` is the only one accepted — anything else fails with a plain-text explanation. Without the overwrite flag, an existing key is **never** touched: the open fails and the report says why. The flag must be exactly `overwrite=1`; near-misses like `overwrite=true` are ignored, deliberately — replacing a key is the kind of thing you should have to spell correctly. An overwrite writes both new files to temporary names and renames them into place, so a failure cannot leave a mismatched pair.

23.2 The report

The open does the work; a read then returns a small machine-parseable report, CR/LF line endings:

```
OK SSH.KEYGEN
TYPE ed25519
OVERWRITE 0
PRIVATE /.ssh/id_ed25519
PUBLIC /.ssh/id_ed25519.pub
```

On failure the report is a single line beginning `SSH.KEYGEN error:` — `key already exists`, `cannot create .ssh directory`, `key generation`, or `unsupported algorithm: <name>` — and the status error is 144. Writes to this channel are refused. Status always reports connected; read the report, close, done.

23.3 Example

```c
/* fuji-keygen: mint the FujiNet's SSH identity (idempotent). */
#include <stdio.h>
#include "fujinet-network.h"

char *url = "n:ssh.keygen://ed25519/";
uint8_t buf[256];

int main(void)
```

```
{
    int16_t n;

    if (network_init() != FN_ERR_OK) return 1;
    network_open(url, OPEN_MODE_READ, OPEN_TRANS_NONE);
    /* even a failed generate leaves a readable one-line report */
    while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
    network_close(url);
    return 0;
}
```

Listing 16. `fuji-keygen.c` — “key already exists” is a fine outcome; parse the first word of the report.

CHAPTER 24

SSH.COPYID

SCHEME	**SSH.COPYID**	CLASS	NetworkProtocolSSHCopyId
FAMILY	Utility (NetworkProtocol)	PORT	22
LOGIN	URL `user:pass@` — the password is required; that is the point		
DOES	install the FujiNet's public key into the server's authorized_keys, exactly like the desktop `ssh-copy-id`		

The bootstrap step: you have a password today and want to stop using it. One open logs into the server **with the password**, appends the FujiNet's public key to `~/.ssh/authorized_keys` (creating directory and file with correct permissions, skipping the append if the key is already present), and reports. From then on, password-less `SSH://user@host` and `SFTP://user@host` devicespecs just work.

24.1 Devicespec

```
N:SSH.COPYID://user:password@host[:port]/
```

Missing username or password fails with 212 before touching the network. The adapter requires the public key from Chapter 23 to exist and look like an ed25519 key; a server that has password login disabled entirely is reported in plain text — you will need to install the key some other way.

24.2 The report

```
OK SSH.COPYID
HOST deepthought
USER fuji
KEY /.ssh/id_ed25519.pub
STATUS installed
```

`STATUS` reads `installed` or `already-present`. Failures are one-line `SSH.COPYID error:` reports — `missing SSH username`, `password required`, `missing hostname`, `public key not found`, `invalid public key`, `server does not allow password authentication`, `remote install failed` — with status error 212 for the credential cases and 144 otherwise.

24.3 Example — the whole passwordless arc

```
OPEN "N:SSH.KEYGEN://ed25519/"            READ -> OK SSH.KEYGEN   CLOSE
OPEN "N:SSH.COPYID://fuji:last-password@deepthought/"
                                          READ -> STATUS installed  CLOSE
OPEN "N:SSH://fuji@deepthought/"          ...a shell, no password, forever...
```

Three opens, run once, from any language in Part VI — after which every SSH and SFTP example in this book works without a secret in sight.

CHAPTER 25

CLIPBOARD

SCHEME	**CLIPBOARD**	**CLASS**	NetworkProtocolClipboard
FAMILY	Utility (NetworkProtocol)	**PORT**	none — storage inside the FujiNet
LOGIN	none		
DOES	read/write the FujiNet clipboard (index 0) and read its nine remembered snippets (1–9); shared with the web UI; 64 KB per snippet		

The clipboard is the missing bridge between your desk and your machine: paste a URL, a listing, or a wall of text into the FujiNet's web UI from your modern browser, and the vintage machine reads it as a file — or write from the machine and copy it out in the browser. The FujiNet remembers the last ten snippets; index 0 is "the clipboard now," 1–9 the history, newest first.

25.1 Devicespec

```
N:CLIPBOARD:///        N:CLIPBOARD:///3        N:CLIPBOARD:///0?binary=1
```

The optional digit selects the snippet — equivalent in the host or path position. Only index 0 may be written (anything else: 167); a history slot that has never been filled reads as 170. `?binary=1` marks written content binary: stored verbatim, never translated by anyone afterward.

25.2 Reference

Open modes: 4 read, 8 write (replaces), 9 append, 12 both. Text is stored LF-terminated internally, so the **stored** side always looks like LF regardless of your aux2 — translation applies between the store and your machine in both directions, and it is applied when the channel **closes** (commit), so a line ending split across two writes still translates correctly. Reads that come up short are NUL-padded to the requested length with 136 raised — check `avail` first, as always. Oversized writes fail with 162 at the 64 KB cap.

PITFALL
A write-mode open **commits at close even if you wrote nothing** — open mode 8 plus an immediate close empties the clipboard. That makes "clear the clipboard" a two-command idiom, and an accidental open destructive. Append (mode 9) never has this hazard.

25.3 Examples

```
(paste a URL into the web UI clipboard, then:)
OPEN    aux1=4 aux2=0   "N:CLIPBOARD:///"
STATUS                  -> avail=41
READ 41                 -> "https://ftp.pigwa.net/stuff/Jumpman.atr"
CLOSE
```

```c
/* clip-note: put text on the clipboard for the browser to collect. */
#include <string.h>
#include "fujinet-network.h"
```

```
char *url = "n:clipboard:///";
char *msg = "score to beat: 12400 (MOLLY)\n";

int main(void)
{
    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_WRITE, OPEN_TRANS_LF) != FN_ERR_OK)
        return 1;
    network_write(url, (uint8_t *)msg, strlen(msg));
    network_close(url);            /* the commit happens here */
    return 0;
}
```

Listing 17. `clip-note.c` — after this runs, the text is sitting in the web UI, ready to copy on any device in the house.

CHAPTER 26

CPM

SCHEME	**CPM**	**CLASS**	`NetworkProtocolCPM`
FAMILY	Utility (NetworkProtocol)	**PORT**	`none — the machine is inside the FujiNet`
LOGIN	none		
DOES	boot a CP/M 2.2 system (RunCPM) inside the FujiNet; the channel is its console — reads are its screen, writes its keyboard		

The strangest devicespec in the book: open it and a Z80 wakes up. `N:CPM://` boots RunCPM — a full CP/M 2.2 with its CCP command processor — inside the FujiNet, with the channel as its console. Point any terminal program at it and you have WordStar-era computing as a peripheral; drives A: onward live on the SD card, so software and files persist between sessions.

26.1 Devicespec

```
N:CPM://
```

Everything after the scheme is ignored — there is nothing to address; the machine is the machine. Open with mode 12 and a translation that matches your terminal (CP/M is a CR/LF world; mode 0 if your terminal handles that itself).

26.2 Reference

Reads return whatever the console has printed (an empty poll reports 202, "nothing yet" — poll status instead); writes are keystrokes. A CP/M **warm boot** (`^C` at the prompt, or a program exiting) restarts the CCP invisibly, exactly as real hardware would. When the session ends for good, status reports 136. Closing the channel halts the emulated machine — the adapter injects a CTRL-C to unstick any pending console read, then tears the Z80 down. Writing while the machine is not running reports 207.

26.3 Example

```
OPEN    aux1=12 aux2=0  "N:CPM://"
READ    ...             -> "RunCPM Version ..." ... "A>"
WRITE   "DIR\r\n"
READ    ...             -> the A: directory
WRITE   "MBASIC\r\n"    ...it is 1981 in there; enjoy...
CLOSE
```

CHAPTER 27

TEST

SCHEME	**TEST**	**CLASS**	`NetworkProtocolTest`
FAMILY	Utility (NetworkProtocol)	**PORT**	`none`
LOGIN	none		
DOES	returns a fixed line of test data; logs and hex-dumps everything — the loopback for translation debugging and the skeleton for new adapters		

TEST answers every open with one fixed line:

```
This is test data being initialized in the test protocol, a single line
of input with line ending.
```

— terminated according to the translation mode you passed (`$9B` , CR, LF, or CR/LF; PETSCII appends nothing). Reads serve that buffer; writes are hex-dumped to the FujiNet's debug log before and after translation. That makes it two tools in one: a loopback that proves your open/ read/status plumbing without a network, and a translation microscope — write a line, watch the log, see exactly what your aux2 did to it.

27.1 The skeleton

The source pair `Test.h` / `Test.cpp` is the official "start here" for a new protocol adapter, and the shape of what you must provide is exactly the contract Part I described: a constructor taking the three buffer pointers; `open` , `close` , `read` , `write` , `status` , and `available` . Register your scheme in `ProtocolParser.cpp` , add the header, and every FujiNet platform — and every chapter in Part VI — can suddenly speak your protocol. That symmetry is the whole architecture, seen from the inside.

27.2 Example

```
OPEN    aux1=4 aux2=1   "N:TEST://anything/"
STATUS                  -> avail=100
READ 100                -> the test line, CR-terminated
WRITE   "HELLO"         -> (watch the debug log hex dump)
CLOSE
```

PART V

Mail and Calendar

Four adapters and two shared frameworks that put a mailbox and a calendar behind the devicespec: folders, messages, and attachments mapped onto the path; day, week, month, and agenda views rendered for your screen width; and — newest of all — sending mail, replying in thread, and creating and editing calendar events from an 8-bit machine.

CHAPTER 28

The Mailbox Model

GMAIL and IMAPS are one machine with two engines. The shared base class maps a mailbox onto the devicespec path, renders everything your machine reads, and parses everything it writes; the providers fetch — and, where they can, send. Everything in this chapter is true of both, with one exception called out where it matters: only GMAIL can send.

28.1 The path is the query

Path	aux1	You get
`/FOLDER`	4	the folder's message count, as a number and line ending
`/FOLDER`	6	the **message index** — subjects, senders, dates
`/FOLDER/N`	4	message N's body (its primary text part)
`/FOLDER/N`	6	message N's **attachment index**
`/FOLDER/N/A`	4	attachment A's raw data (A = 0 is the body again)
`/`	8	**compose** a new message (write, then close)
`/FOLDER/N`	8	**reply** to message N

Table 20. The mailbox grammar. N is the message's position in the folder — the providers define which end is 1. Any other path depth is 165; append and read-write are refused with 135; write works only on providers that can send — GMAIL yes, IMAPS no.

Two query parameters page the index: `?range=START-END` (inclusive, 0-based positions in the listing order) and `?newest=1` (default — newest first) or `?newest=0` for oldest-first. Without a range you get the first 20; the ceiling per listing is 200.

28.2 aux2: width by day, structs by night

For a **message body** read, aux2 is the ordinary translation mode of Part I. For the **index** opens, it is something better:

- **aux2 = 255** — the index arrives as packed binary records, made for programs.
- **any other value** — a human-readable listing, rendered at a line width of aux2′s low seven bits; 0 (and NOS's habitual 128) means the platform default — 38 columns on Atari, 40 on Apple II, 32 on ADAM and CoCo, 80 on RS-232 machines.

The human message index is built with real 8-bit craft: each message is two physical lines — flag, number, sender, and date on the first; subject indented on the second — sized so a 40-column screen wraps exactly at the seam and an 80-column screen reads the same bytes as one line. The attachment index is a simple `# name type size` table.

The binary records, little-endian, NUL-padded fixed fields:

msgNum u32	displayName char[32]	emailAddress char[48]	subject char[128]	timestamp u64

`MailIndexItem` — 220 bytes per message. The attachment record (`MailAttachmentItem` , 313 bytes) is `attachmentNum` u8, `displayName` char[128], `fileName` char[128], `mimeType` char[48], `length` u64.

28.3 Reading rhythm

Everything is fetched and staged at open — the count, the index, the body, the attachment — so status tells you the total at once and reads simply drain it to 136. There is no pagination inside one open; a new range means a new open. Which end of the mailbox is message 1, and what the folder names are, belongs to the provider chapters.

28.4 Writing a message

Where the provider allows it, mode 8 turns the machine around: a write-mode open starts a **draft**, you write header lines and a body, and the close **sends** — one shot, atomically, with the verdict in the next status. The draft is the mail you already know, RFC822 with the flour still on it:

```
TO: alice@example.com
CC: bob@example.com
SUBJECT: Hello from an Atari

Sent from my 130XE over FujiNet.
```

Header lines are `KEY: value` — the colon is required — with case-insensitive keys and trimmed values, ended by any line ending your machine likes (`$9B` , CR, LF, or CRLF, freely mixed; aux2 translation is never applied to a write channel, so what you write is exactly what the parser reads). Exactly four keys exist: `TO` , `CC` , `BCC` , and `SUBJECT` . `TO` , `CC` , and `BCC` **accumulate** — repeat them to add recipients — while a repeated `SUBJECT` is last-wins. Anything else rejects the draft, and one absence is deliberate: there is no `FROM` , because the sender is the authenticated account, and an 8-bit machine does not get to forge it. The first blank line ends the headers and is consumed; everything after it is the body, verbatim — a colon there is just a colon.

Replying (write mode on `/FOLDER/N`) fills the blanks for you: an omitted `TO` goes to the original's `Reply-To` (or, failing that, its `From`), and an omitted `SUBJECT` becomes `Re:` the original's — never doubled into `Re: Re:` . Anything you do write wins over the defaults. The shortest legal reply is a blank line and a sentence. The target is resolved and pinned at open, so mail arriving between your open and your close cannot renumber it out from under you; a message number past the end of the folder fails the open with 170.

The close's verdict arrives as the channel error in the next status: 1 for sent; 132 for a rejected draft — an unknown key, a header line without a colon, no `TO` and no default to fall back on — with the specific reason named only in the debug log; 162 if the draft exceeded 16 KB (nothing is sent). **Writing nothing at all is a clean abort**: open-then-close sends nothing and errs nothing. Reading a write channel answers 131, and its status never shows EOF — a draft is not a thing you drain. What goes out is `text/plain` in UTF-8, one part: there is no attachment on the sending side, and sending is the **only** write there is — nothing is ever marked read, moved, or deleted. (On the SIO, AdamNet, DriveWire, and RS-232 buses the commit's verdict is latched through the close into the next status; the other bus layers do not yet latch it — Appendix E.)

CHAPTER 29

GMAIL

SCHEME	GMAIL	CLASS	NetworkProtocolGMAIL
FAMILY	Mailbox (NetworkProtocolMailbox)	PORT	443 (Gmail REST API)
LOGIN	OAuth — the same Google grant as GDRIVE; scopes gmail.readonly + gmail.send		
DOES	read the mailbox: counts, indexes, bodies, attachments · compose new mail · reply in-thread; folders are Gmail labels		

GMAIL speaks to your Gmail over Google's REST API, using the same one-time web-UI authorization as GDRIVE and GCAL — with the mail scopes included in the grant. Reading is still scrupulously hands-off: nothing is ever marked read, moved, or deleted, which is exactly what you want wired to a machine that cannot show an unsubscribe link. But since the newest firmware the adapter is no longer only a viewer — it can **send**: compose a fresh message, or reply to one it just showed you, threaded where the original lives.

29.1 Devicespec

```
GMAIL:///Inbox          GMAIL:///Inbox/3          GMAIL:///Work/1/2
```

The folder is a Gmail **label**, matched case-insensitively — `Inbox`, `Sent`, `Starred`, or any label you created. Message 1 is the **oldest**; the newest message's number equals the folder's count — so "read the latest" is: read the count from `/Inbox`, then open `/Inbox/<count>`. (The index listing still shows newest first by default; `?newest=0` flips it.) Bodies prefer the plain-text part and fall back to HTML; attachments are any parts with filenames, indexed from 1. For writing, the same two shapes in mode 8: `GMAIL:///` composes, `GMAIL:///Inbox/3` replies to message 3.

29.2 Sending

The draft grammar is Chapter 28's; what GMAIL adds is where it goes. A committed compose is handed to Gmail's `messages.send`: Google stamps the `From` and the date from the authenticated account, files a copy under `Sent`, and delivers. A committed reply is threaded into the original conversation — the adapter carries the original's message id and references, so real mail clients see a proper reply. One caveat worth knowing: Gmail groups a conversation by subject as well as by thread, so if you override the default `Re:` subject, your reply may display outside the conversation it technically belongs to.

Sending needs the `gmail.send` scope, which joined the shared Google grant alongside `gmail.readonly`. A grant issued before a scope existed never gains it retroactively, so an old authorization keeps reading happily and fails its first send with 167 — re-authorize Google in the web UI and it heals.

API answer	Code	Reported as
401 — token invalid/expired	212	INVALID_USERNAME_OR_PASSWORD — re-authorize Google in the web UI
403 — scope missing	167	ACCESS_DENIED — the grant predates the mail scopes (`gmail.send` , for a write); re-authorize
404 — no such label or message	170	FILE_NOT_FOUND
no answer / 5xx	210	SERVICE_NOT_AVAILABLE
anything else	144	GENERAL

Table 21. GMAIL error mapping.

29.3 Examples

```
OPEN    aux1=4 aux2=0   "GMAIL:///Inbox"        READ -> "217"
OPEN    aux1=6 aux2=0   "GMAIL:///Inbox?range=0-9"
READ    ...             -> ten newest, two lines each, your screen width
OPEN    aux1=4 aux2=2   "GMAIL:///Inbox/217"    READ -> the newest body
```

```
OPEN    aux1=8 aux2=0   "GMAIL:///"             (compose)
WRITE   "TO: vi@example.com\x9bSUBJECT: 7pm?\x9b"
WRITE   "\x9bpizza night. come.\x9b"
CLOSE                   -> next STATUS: err=1, sent
OPEN    aux1=8 aux2=0   "GMAIL:///Inbox/217"    (reply to the newest)
WRITE   "\x9bon my way.\x9b"
CLOSE                   -> err=1, sent — and threaded
```

```c
/* gmail-latest: print the newest message in the inbox. */
#include <stdio.h>
#include <stdlib.h>
#include "fujinet-network.h"

uint8_t buf[512];
char url[64];

int main(void)
{
    int16_t n;
    int count;

    if (network_init() != FN_ERR_OK) return 1;

    network_open("n:gmail:///Inbox", OPEN_MODE_READ, OPEN_TRANS_NONE);
    n = network_read("n:gmail:///Inbox", buf, sizeof(buf) - 1);
    network_close("n:gmail:///Inbox");
    if (n <= 0) return 1;
    buf[n] = 0;
    count = atoi((char *)buf);

    sprintf(url, "n:gmail:///Inbox/%d", count);   /* newest = count */
    if (network_open(url, OPEN_MODE_READ, OPEN_TRANS_LF) != FN_ERR_OK)
        return 1;
    while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
```

```
    network_close(url);
    return 0;
}
```

Listing 18. `gmail-latest.c` — count first, then open the highest-numbered message.

```c
/* gmail-send: compose and send a message. */
#include <string.h>
#include "fujinet-network.h"

char *url = "n:gmail:///";
char *draft =
    "TO: alice@example.com\n"
    "SUBJECT: Greetings from 1979\n"
    "\n"
    "This message left the machine by way of\n"
    "a FujiNet. No stamp required.\n";

int main(void)
{
    uint16_t bw; uint8_t conn, err;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_WRITE, OPEN_TRANS_NONE) != FN_ERR_OK)
        return 1;
    network_write(url, (uint8_t *)draft, strlen(draft));
    network_close(url);                 /* the send */

    network_status(url, &bw, &conn, &err);
    return err == 1 ? 0 : 1;            /* 1 = sent */
}
```

Listing 19. `gmail-send.c` — headers, a blank line, a body; the close is the send and the next status is the verdict. aux2 is ignored on a write open, so the trans mode is decoration: the parser takes any line ending as written.

```c
/* gmail-reply: answer the newest message in the inbox. */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include "fujinet-network.h"

uint8_t buf[32];
char url[64];

int main(void)
{
    int16_t n;
    uint16_t bw; uint8_t conn, err;

    if (network_init() != FN_ERR_OK) return 1;

    network_open("n:gmail:///Inbox", OPEN_MODE_READ, OPEN_TRANS_NONE);
    n = network_read("n:gmail:///Inbox", buf, sizeof(buf) - 1);
    network_close("n:gmail:///Inbox");
    if (n <= 0) return 1;
    buf[n] = 0;

    sprintf(url, "n:gmail:///Inbox/%d", atoi((char *)buf));
```

```
        if (network_open(url, OPEN_MODE_WRITE, OPEN_TRANS_NONE) != FN_ERR_OK)
            return 1;                       /* a bad N fails here: 170 */
        network_write(url, (uint8_t *)"\nGot it -- thanks!\n", 19);
        network_close(url);                 /* the send */

        network_status(url, &bw, &conn, &err);
        return err == 1 ? 0 : 1;
    }
```

Listing 20. `gmail-reply.c` — the leading blank line means “no headers”: `TO` and `SUBJECT` come from the message being answered, and the reply lands in its thread.

CHAPTER 30

IMAPS

SCHEME	IMAPS	CLASS	NetworkProtocolIMAPS
FAMILY	Mailbox (NetworkProtocolMailbox)	PORT	993 (implicit TLS)
LOGIN	URL `user:pass@`, else the stashed username/password specials		
DOES	read-only mailbox for any IMAP server over TLS — Fastmail, a self-hosted Dovecot, an office server		

IMAPS is the same mailbox as GMAIL for everyone whose mail is **not** Gmail: it speaks IMAP4 to any server over implicit TLS. No OAuth, no relay — a username and password of the ordinary kind. (For Gmail itself, prefer the GMAIL adapter; Google dislikes password IMAP.)

30.1 Devicespec

```
IMAPS://user:pass@mail.example.com/INBOX/3
```

Everything after the host follows the mailbox grammar; the folder is the IMAP mailbox name (`INBOX` is universal; subfolders use the server's own separator). Message numbers **are** IMAP sequence numbers: 1 is the oldest, the count is the newest — the same arithmetic as GMAIL. The connection is TLS from the first byte on port 993; there is no STARTTLS mode, so a server offering only port 143 is out of reach. Sending is not among IMAPS's talents either: a mode-8 open is refused with 135 — Chapter 28′s compose grammar is GMAIL-only for now.

Condition	Code	Reported as
no host in the URL	165	INVALID_DEVICESPEC
no username	212	INVALID_USERNAME_OR_PASSWORD
TLS connect failed	200	CONNECTION_REFUSED
server silent	202	SOCKET_TIMEOUT
greeting not OK	210	SERVICE_NOT_AVAILABLE
LOGIN rejected	212	INVALID_USERNAME_OR_PASSWORD
no such folder / message / attachment	170	FILE_NOT_FOUND
fetch or parse failure	144	GENERAL

Table 22. IMAPS error mapping — the most fine-grained login diagnostics in the book, worth surfacing to your user.

30.2 Example

```c
/* imap-count: how many messages are waiting on the family server? */
#include <stdio.h>
#include "fujinet-network.h"

char *url = "n:imaps://fuji:sekrit@mail.example.com/INBOX";
uint8_t buf[32];

int main(void)
{
    int16_t n;

    if (network_init() != FN_ERR_OK) return 1;
```

```
        if (network_open(url, OPEN_MODE_READ, OPEN_TRANS_NONE) != FN_ERR_OK)
            return 1;
        n = network_read(url, buf, sizeof(buf) - 1);
        if (n > 0) { buf[n] = 0; printf("%s messages\n", buf); }
        network_close(url);
        return 0;
    }
```

Listing 21. `imap-count.c` — the count open, the cheapest possible mail check; wire it to a status bar.

CHAPTER 31

The Calendar Model

GCAL and ICAL share a framework the way the mail providers do — but the calendar base does more: it computes date windows (correctly across DST), expands and sorts events, renders four views, caches the last window, and — provider permitting — accepts **written** events back. This chapter is that machinery; the two after it are the engines.

31.1 The path: what, when, which

```
SCHEME://selector/VIEW[/DATE[/N]]
```

selector which calendar — a name, an id, empty for the provider's default set. Its exact meaning is the provider's.
VIEW `DAY`, `WEEK`, `MONTH`, or `AGENDA` (upcoming events from now), matched case-insensitively. The keyword is found by scanning the path **from the end**, so a calendar whose own name contains `day` still parses.
DATE `YYYY-MM-DD` (or `YYYY-MM` for MONTH); omitted means today.
N 1-based event number within that view's listing.

Path	aux1	You get
`/`	6	the list of calendars (or 4 for their count)
`/sel/VIEW[/DATE]`	6	the event index for the period
`/sel/VIEW[/DATE]`	4	the event count for the period
`/sel/VIEW/DATE/N`	4	event N in full detail
`/sel`	8	**compose** a new event (write, then close)
`/sel/VIEW/DATE/N`	8	**edit** event N

Table 23. The calendar grammar. Mode 12 is accepted as read (some buses default to it); append is refused; write works only on providers that allow it — GCAL yes, ICAL no.

Query parameters: `?category=name` filters; `?count=` caps an AGENDA (20 by default) and `?days=` sets its horizon (365 by default); `?wkst=MO` starts weeks on Monday; `?tz=` overrides the FujiNet's configured timezone for both parsing and display. And `?view=`, `?date=`, `?n=` bypass path scanning entirely — the escape hatch for pathological calendar names.

Event numbering is deterministic: the window's events are sorted by start time (then end, then identity) and numbered from 1, so the N you saw in an index listing means the same event in a detail or edit open moments later. Recurring events are expanded into their occurrences before numbering; each occurrence is its own N. A two-minute cache makes the index-then-detail pattern cost one provider round-trip.

31.2 Rendering

aux2 works exactly as in the Mailbox chapter for index opens — 255 for packed records, otherwise a width — and is **ignored** for detail reads: every byte of calendar text is composed by the adapter, so translation is always off. The human index is one event per line — start time (or `all-day`), a date column whose format fits the view (none for DAY, `Fri` for WEEK, `Fr 28` for MONTH, `28 Aug` for AGENDA), then the summary — followed by an indented location line when there is one. The packed record:

event-Num u32	start u64	end u64	flags u8	summary char[96]	location char[64]	category char[32]	uid char[64]

`CalEventItem` — 277 bytes; times are Unix epoch UTC, end exclusive; flags: bit 0 all-day, bit 1 recurring. The calendar-list record (`CalListItem`, 224 bytes) is `name` char[64], `category` char[32], `id` char[128].

31.3 Writing an event

A write-mode open starts a **draft**; you write field lines; the close commits — one shot, atomically, with the result in the next status. Field lines are `KEY: value` — the colon is required — with case-insensitive keys, trimmed values, and any line ending your machine likes (`$9B`, CR, LF, or CRLF, freely mixed; as with mail, aux2 translation is never applied to a write channel). Blank lines are skipped — unlike a mail draft, there is no header/body divide:

```
SUMMARY: Dentist
START: 2026-09-03 14:30
END: 2026-09-03 15:15
LOCATION: 12 Main St.
DESCRIPTION: bring the x-rays
DESCRIPTION: and the insurance card
CATEGORY: health
```

Six keys exist: `SUMMARY`, `START`, `END`, `LOCATION`, `DESCRIPTION`, and `CATEGORY`. `DESCRIPTION` repeats to build paragraphs; the other duplicate keys are last-wins; an unknown key rejects the draft, so a typo cannot silently drop data. Times take `YYYY-MM-DD` for a date and `YYYY-MM-DD HH:MM[:SS]` for a moment — a `T` may stand where the space is, the compact `YYYYMMDD[THHMMSS]` forms work too, and a trailing `Z` or `±HH:MM` offset makes the value absolute. Without one, the time floats: it is resolved in the request's `?tz=` timezone, default the FujiNet's own.

Composing requires `SUMMARY` and `START`; a missing `END` defaults to one hour (or one day for an all-day event — a `START` with no time part means all-day, and an all-day `END` names the **last day, inclusive**, the way humans talk). **Editing** (write mode on `/…/N`) addresses event N exactly as the index numbered it, resolved and pinned at open — a number past the period's count fails the open with 170 — and changes only the fields you send. With neither `START` nor `END`, the times are not touched at all; a lone `START` moves the event and keeps its duration; a lone `END` stretches it, but must stay in the event's form. Switching between all-day and timed requires sending `START` in the new form (the compose defaults then supply the length).

The commit's verdict arrives as the channel error after close: 1 for created/updated; 132 for a rejected draft — an unknown key, a line without its colon, an unparseable time, a missing `SUMMARY` or `START` on compose, an `END` at or before its `START`, or an all-day date paired with a timed one, the specific reason named only in the debug log; 162 if the draft exceeded 16 KB (nothing is sent); 170 if the edit target vanished between open and close; and **writing nothing at all is a clean abort** — open-then-close composes nothing and errs nothing. There is no delete: these adapters can put an event on the calendar and move it, but taking one off still belongs to a bigger machine. A successful commit also drops the two-minute listing cache, so the next index open renumbers freshly — re-list before you edit again. (The verdict is latched through the close into the next status on the SIO, AdamNet, DriveWire, and RS-232 buses; the other bus layers do not yet latch it — Appendix E.)

CHAPTER 32

GCAL

SCHEME	GCAL	CLASS	NetworkProtocolGCAL
FAMILY	Calendar (NetworkProtocolCalendar)	PORT	443 (Google Calendar API)
LOGIN	OAuth — the same Google grant; calendar.readonly to read, calendar.events to write		
DOES	read every calendar on the account, merged or singly · compose new events · edit existing ones		

GCAL is the family calendar on the kitchen-table Atari: every Google calendar your account can see, rendered for a 40-column screen — and since the newest firmware, a machine that can **add** the dentist appointment, not just display it. Authorization is the same single Google grant as GDRIVE and GMAIL.

32.1 Devicespec

```
GCAL:///DAY          GCAL:///Work/WEEK          GCAL:///*/MONTH/2026-09
```

The selector resolves in a forgiving order: calendar **name** (case-insensitive), then calendar **id**, then it is passed to Google verbatim — so `primary` and raw ids work without a lookup. An **empty** selector merges every calendar you have marked shown in Google's UI (each event's category becomes its calendar's name); `*` merges everything, shown or not; up to eight calendars per merge. Each event's category is chosen usefully: an explicit category set through this adapter wins, then Google's color name (`Tomato`, `Sage`, `Peacock`…), then the calendar's name — so `?category=` filtering has something to grab in every calendar.

32.2 Writing

Compose targets one calendar: an empty selector means `primary`, a name or id means that calendar, and `*` is refused (165) — "everywhere" is not a place to put an event. One naming corner: a calendar literally **named** `Day`, `Week`, `Month`, or `Agenda` cannot be compose-targeted by name, because the view scan claims that path segment — use its id. A `CATEGORY` you write is stored as private extended data on the event — invisible to other Google clients, but exactly where this adapter's reads look first, so your categories round-trip. Edits go up as a Google `PATCH`, translating correctly between Google's two date forms — so sending `START` in the other form really does switch an event between all-day and timed. And because the index expands recurring events into their occurrences, editing N touches **that occurrence only** — never the series.

If the account's grant predates the calendar scopes, reads or writes fail with 167 and the debug log names the problem: re-authorize Google in the web UI — a grant never gains scopes retroactively. Unparseable API responses report 213; everything else follows the HTTP mapping from Chapter 14′s table (with 429 → 210).

32.3 Examples

```
OPEN    aux1=6 aux2=0   "GCAL:///DAY"       READ -> today, merged
OPEN    aux1=4 aux2=0   "GCAL:///Work/WEEK" READ -> "4"
OPEN    aux1=8 aux2=0   "GCAL:///"          (compose to primary)
WRITE   "SUMMARY: Call Vi\x9bSTART: 2026-09-02 19:00\x9b"
CLOSE                   -> next STATUS: err=1, created
```

```
OPEN    aux1=8 aux2=0   "GCAL:///Family/DAY/2026-09-04/2"   (edit)
WRITE   "START: 2026-09-05 18:30\x9b"
CLOSE                   -> err=1, moved — same length, new evening
```

```c
/* gcal-add: put an event on the family calendar. */
#include <string.h>
#include "fujinet-network.h"

char *url = "n:gcal:///Family";
char *draft =
    "SUMMARY: Pizza night\n"
    "START: 2026-09-04 18:30\n"
    "LOCATION: home\n"
    "CATEGORY: fun\n";

int main(void)
{
    uint16_t bw; uint8_t conn, err;

    if (network_init() != FN_ERR_OK) return 1;
    if (network_open(url, OPEN_MODE_WRITE, OPEN_TRANS_LF) != FN_ERR_OK)
        return 1;
    network_write(url, (uint8_t *)draft, strlen(draft));
    network_close(url);                 /* the commit */

    network_status(url, &bw, &conn, &err);
    return err == 1 ? 0 : 1;            /* 1 = created */
}
```

Listing 22. `gcal-add.c` — write the draft, close to commit, read the verdict from status.

```c
/* gcal-move: reschedule one event on the family calendar. */
#include <stdio.h>
#include <string.h>
#include "fujinet-network.h"

char *day = "n:gcal:///Family/DAY/2026-09-04";
char url[80];
uint8_t buf[512];

int main(void)
{
    int16_t n;
    uint16_t bw; uint8_t conn, err;
    int ev;

    if (network_init() != FN_ERR_OK) return 1;

    /* list the day, numbered, so the user can pick */
    network_open(day, 6, 0);
    while ((n = network_read(day, buf, sizeof(buf) - 1)) > 0) {
        buf[n] = 0;
        printf("%s", buf);
    }
    network_close(day);

    printf("move which event? ");
```

```
        scanf("%d", &ev);

        sprintf(url, "%s/%d", day, ev);     /* .../DAY/2026-09-04/N */
        if (network_open(url, OPEN_MODE_WRITE, OPEN_TRANS_NONE) != FN_ERR_OK)
            return 1;                       /* a bad N fails here: 170 */
        network_write(url, (uint8_t *)"START: 2026-09-05 18:30\n", 24);
        network_close(url);                 /* the commit */

        network_status(url, &bw, &conn, &err);
        return err == 1 ? 0 : 1;            /* 1 = updated */
    }
```

Listing 23. `gcal-move.c` — an edit is a write open on the event's index address. The lone `START:` moves the event and keeps its length; everything unsent — summary, location, category, even the `END` — stays as it was.

CHAPTER 33

ICAL, WEBCAL, and ICALH

SCHEME	ICAL • WEBCAL • ICALH	CLASS	NetworkProtocolICAL
FAMILY	Calendar (NetworkProtocolCalendar)	PORT	443 (ICAL, WEBCAL) · 80 (ICALH)
LOGIN	none — feeds are their own secret		
DOES	read-only views over any published .ics feed — school schedules, team fixtures, holidays, any "subscribe to calendar" link		

Half the world's calendars are not in anyone's account — they are published `.ics` feeds: the school year, the league fixtures, the public holidays, the phase of the moon. These schemes point the whole calendar framework at any such feed. `ICAL` and `WEBCAL` fetch over HTTPS — `WEBCAL` so you can paste a `webcal://` subscription link unchanged — and `ICALH` is the plain-HTTP escape for LAN feeds.

33.1 Devicespec

```
ICAL://calendar.google.com/calendar/ical/en.usa%23holiday@group.v.calendar.google.com/public/basic.ics/MONTH
```

Everything between the scheme and the view keyword is the feed's host and path, resent **verbatim** — percent-escapes and all, which matters because private feed URLs embed secrets that must survive unmangled. After it, the usual `/VIEW[/DATE[/N]]`, and every query parameter from the model chapter (`?tz=` earns its keep here: feeds that name exotic timezones are interpreted in your configured zone unless told otherwise).

33.2 Reference

The feed is parsed **streaming**: events are matched against the requested window as they pass, recurring rules are expanded only within it, and detached-occurrence overrides are patched in at the end of the feed — so a season-long feed costs window-sized memory, and pathological feeds hit safety rails (a line cap, an expansion cap) rather than the FujiNet's RAM. Practical consequences of feed-hood: there is no calendar **list** (a `/` open reports 165 — a feed is one calendar); every write reports 135; and the detail open re-scans the feed a second time (with the window widened a day each side so edge events still match) to pick up the `DESCRIPTION` the index pass skipped. One firm requirement: the adapter never advertises compression, and a server that compresses anyway is refused with 146 rather than misparsed.

33.3 Example

```
/* holidays: what's the next public holiday? */
#include <stdio.h>
#include "fujinet-network.h"

char *url = "n:ical://www.thunderbird.net/media/caldata/autogen/"
            "UnitedStatesHolidays.ics/AGENDA?count=1";
uint8_t buf[256];

int main(void)
{
    int16_t n;
```

```
        if (network_init() != FN_ERR_OK) return 1;
        if (network_open(url, 6, 0) != FN_ERR_OK)   /* index, default width */
            return 1;
        while ((n = network_read(url, buf, sizeof(buf) - 1)) > 0) {
            buf[n] = 0;
            printf("%s", buf);
        }
        network_close(url);
        return 0;
    }
```

Listing 24. `holidays.c` — AGENDA with `?count=1` is “the next thing that happens,” a one-line hello-world for any feed.

PART VI

From Your Machine

Six dialects for one vocabulary. Each chapter shows how the protocols of Parts II–V are driven from one programming environment — the commands, the idioms, the platform's own sharp edges — and ends with a pointer to the full programmer's guide, where the deep material lives. The protocols themselves never change; only the spelling does.

CHAPTER 34

Atari BASIC

Atari BASIC reaches the network device through the **N: handler** (NDEV), a small relocatable CIO driver loaded at boot — `AUTORUN.SYS` on the boot disk, or `NDEV.COM` under a command-line DOS. Once resident, `N:` is a device like `D:` or `P:`, and the whole of this book is available through five BASIC verbs and `XIO`.

34.1 The verbs

```
10 OPEN #1,4,0,"N:HTTPS://FUJINET.ONLINE/HELLO.TXT"
20 STATUS #1,A:BW=PEEK(746)+PEEK(747)*256
30 IF BW=0 AND PEEK(748)=1 THEN 20
40 TRAP 80:FOR I=1 TO BW:GET #1,C:? CHR$(C);:NEXT I:GOTO 20
80 CLOSE #1
```

`OPEN`'s two numeric arguments are exactly Part I's aux1 and aux2. After a `STATUS #ch,A`, the four status bytes of Chapter 5 sit in the `DVSTAT` locations: bytes waiting at `PEEK(746)+PEEK(747)*256`, connection at `PEEK(748)`, the error code at `PEEK(749)`. `INPUT` / `PRINT` move lines, `GET` / `PUT` move bytes; a trailing `;` on `PRINT #1` suppresses the EOL when a protocol should not receive one.

34.2 XIO is the command table

The handler's design rule makes Chapter 6 directly usable: **the XIO number is the command byte.** `XIO 32` renames, `XIO 42` makes a directory, `XIO 65` accepts a TCP client, `XIO 252,#1,0,1,"N:"` sets channel mode JSON — read the decimal column of Chapter 6's table and you have the Atari reference. For commands it has never heard of, the handler asks the FujiNet for the data direction (the `$FF` inquiry) and relays; the JSON pattern from the guide:

```
20 OPEN #1,4,0,"N:HTTPS://api.example.com/status.json"
30 XIO 252,#1,0,1,"N:"        :REM CHANNEL MODE = JSON
40 XIO 80,#1,0,0,"N:"         :REM PARSE
50 XIO 81,#1,0,0,"N:/status/message"
60 INPUT #1,V$ : PRINT V$
70 CLOSE #1
```

One XIO is local: `XIO 15` flushes the handler's transmit buffer immediately — NDEV normally holds bytes until an EOL — and never crosses the bus.

34.3 The platform's sharp edges

- **Reads cap at 127 bytes** per call — the loop above never notices, but a `bw` larger than 127 takes several `GET` rounds.
- **Going to the DOS 2 menu unloads the handler** (DUP overwrites it); return via a program that reloads NDEV, or use a command-line DOS.
- NDEV speaks only to the network device — mounting hosts and drives (the Fuji device) is out of its reach, which is NOS's department.
- The HTTP header specials (`$4D`) are not in the handler's inquiry table and report error 146; full header work needs direct SIO or NOS's world, per the guide.

THE FULL GUIDE

FujiNet Programming Guide for the Atari — `fujinet-manuals/atari/programmers_guide/` — the N: device from BASIC and from assembly, the complete SIO reference for direct access, the NDEV source listing, Action! and Logo bindings, and the traditional netcat.

CHAPTER 35

Applesoft BASIC

On the Apple II, the FujiNet extension is a **BASIC.SYSTEM external command** set: `BRUN FUJINET` hooks the ProDOS command parser at `EXTRNCMD`, and eighteen new `N`-commands become part of the language — typed bare at the `]` prompt or issued from a program with the classic `PRINT CHR$(4);"…"` idiom. (It is deliberately **not** an ampersand extension: Applesoft tokenizes `NREAD` into `N`-plus-`READ`-token before `&` ever runs, so the command hook is the only clean road.)

35.1 The command set

Command	Does
`NOPEN ch,spec,mode,trans`	open — the four-beat begins
`NCLOSE ch` · `NSTATUS ch,bw,conn,err`	close; status into three variables
`NREAD ch,v$,n` · `NWRITE ch,data$,n`	move bytes through strings
`NJSONPARSE ch` · `NJSONQUERY ch,v$,query`	the JSON three-beat, mode switch included
`NCD` `NCAT` `NCATALOG` `NMKDIR` `NRMDIR` `NDEL` `NTYPE`	filesystem one-shots, each taking a devicespec
`NLOAD spec` · `NSAVE spec`	Applesoft programs straight off and onto the network
`NTRANS ch,n` · `NACCEPT ch` · `NLOGIN ch,user\$,pass\$` · `NHTTPMODE ch,n`	sticky translation; TCP accept; credentials; HTTP channel mode

Table 24. The eighteen network commands (plus `CALL 32771` / `32774` for the network printer). Channels run 1–15; 0 is the extension's own.

Modes and translations are Part I's values, with one addition: `trans` 4 is PETSCII. Errors surface the ProDOS way — an `ONERR` handler reads `PEEK(222)`: 3 no device, 6 path not found, 8 I/O error, 16 syntax, 2 range, 14 program too large.

35.2 The idiom, complete

```
10 D$ = CHR$(4): Q$ = CHR$(34)
20 U$ = "N:HTTPS://ICANHAZIP.COM/"
30 PRINT D$;"NOPEN 1,";Q$;U$;Q$;",4,0"
40 PRINT D$;"NSTATUS 1,BW,CN,ER"
50 IF BW = 0 AND CN = 1 THEN 40
70 PRINT D$;"NREAD 1,A$,BW"
80 PRINT "MY IP IS ";A$
90 PRINT D$;"NCLOSE 1"
```

The quote characters around the devicespec keep BASIC.SYSTEM from parsing the URL's own commas and colons. Strings cap transfers at 255 bytes per `NREAD` / `NWRITE` — loop as always. And `NLOAD` / `NSAVE` deserve a sentence of appreciation: a program library on a TNFS server, one command away, from a machine with no network card in it.

THE FULL GUIDE

FujiNet BASIC Extension for the Apple II — `fujinet-manuals/apple2/basic_extension/` — every command with its errors, the memory map, and the full assembly source; and the **FujiNet Programming Guide for the Apple II** — `fujinet-manuals/apple2/programmers_guide/` — for the SmartPort protocol beneath it.

CHAPTER 36

Coleco ADAM SmartBASIC 1.x

The ADAM's road is the most luxurious: seventeen **native statements**, patched into SmartBASIC 1.x itself (the interpreter source survived, so the FujiNet commands are real tokens, not stowaways). Boot the modified SmartBASIC from a data pack or disk image and `NOPEN` is as much a part of the language as `PRINT`.

36.1 The statement set

The vocabulary matches the Apple II chapter almost word for word — `NOPEN d,url$,mode,trans`, `NCLOSE`, `NREAD d,buf$,len`, `NWRITE`, `NSTATUS d,bw,conn,err`, `NJSONPARSE`, `NJSONQUERY`, `NCD`, `NDIR`, `NMKDIR`, `NRMDIR`, `NDEL`, `NLOAD`, `NSAVE`, `NACCEPT`, `NLOGIN`, `NHTTPMODE` — with ADAM spellings: channels 1–15 map onto EOS character devices 9 and up; `NDIR` is the directory command; `NLOAD` / `NSAVE` move ASCII listings in immediate mode.

Errors are BASIC-native: any network failure raises error 37, prints `?Network Error <n>` with the code from Chapter 5′s table, and is catchable with `ONERR` — after which `NSTATUS` retrieves the code. Codes 1 and 136 never raise. A machine with no FujiNet attached fails fast with `?ILLEGAL QUANTITY ERROR` instead of hanging.

36.2 A worked example — JSON from orbit

```
220 ONERR GOTO 900
300 NOPEN 1,"N:HTTP://api.open-notify.org/iss-now.json",4,0
310 NJSONPARSE 1
320 NJSONQUERY 1,"/iss_position/latitude",lat$
330 NJSONQUERY 1,"/iss_position/longitude",lon$
340 NCLOSE 1
350 PRINT "ISS AT ";lat$;" ";lon$
360 END
900 NSTATUS 1,bw,conn,err : PRINT "Network Error: ";err
```

One ADAM-specific grain: AdamNet moves at most 1024 bytes per bus transaction, so a large `avail` drains in kilobyte sips — invisible in BASIC, worth knowing when you time things.

> **THE FULL GUIDE**
> **SmartBASIC 1.x FujiNet appendix** — in the `smartbasic-1.x` repository (with `FUJINET.md`, the patched interpreter source, and runnable examples); and the **FujiNet Programming Guide for the Coleco ADAM** — `fujinet-manuals/adam/programmers_guide/` — for EOS and Z80 assembly access to the same device.

CHAPTER 37

C with fujinet-lib

Every C example in this book has been quietly teaching this chapter: fujinet-lib is the portable C API over the network device, built for cc65, z88dk, CMOC, and friends — one source file compiling for Atari, Apple II, ADAM, CoCo, Commodore, MS-DOS, PMD 85, and RC2014. Its network header is a thin, honest wrapper over Part I: same modes, same channel tricks, same error table underneath.

37.1 The surface

Call	Does
`network_init()`	find the FujiNet; once, at program start
`network_open(spec, mode, trans)`	the open — aux1 and aux2 by their real names
`network_close(spec)`	the close
`network_read(spec, buf, len)`	blocking read → **bytes read, or –error**
`network_read_nb(...)`	non-blocking variant — returns what is ready
`network_write(spec, buf, len)`	write → `FN_ERR_*`
`network_status(spec, &bw, &conn, &err)`	the four status bytes, unpacked
`network_json_parse(spec)` `network_json_query(spec, path, out)`	the JSON three-beat (mode switch included; query → bytes or –error)

`network_http_post/put/delete(...)`
`network_http_start_add_headers/`
`add_header/end_add_headers`
`network_http_set_channel_mode(...)`
	the whole Chapter 14 toolkit

`network_fs_rename/delete/mkdir/` `rmdir/lock/unlock/cd(...)`	the one-shot filesystem specials
`network_ioctl(cmd, a1, a2, spec, ...)`	everything else — any command byte from Chapter 6

Table 25. `fujinet-network.h`, the parts you will use. The devicespec string is the channel handle: pass the same string to every call.

The one convention that bites: **two return styles.** Most calls return an `FN_ERR_*` byte — 0 is success, and `fn_device_error` holds the device's error code from Chapter 5 when you need the real story. But the three data-returning calls (`network_read`, `network_read_nb`, `network_json_query`) return a **count**, negative on failure — so end of file arrives as `-136`, and `while ((n = network_read(...)) > 0)` is the canonical loop. Named constants exist for modes 4/8/12, the HTTP modes, and translations 0–4; directory (6) and append (9) you pass as literals.

37.2 A complete program

```c
/* whatsup: one program, three parts of this book. */
#include <stdio.h>
#include "fujinet-network.h"

uint8_t buf[256];

int main(void)
{
    int16_t n;
```

```
    char *api = "n:https://api.open-notify.org/astros.json";
    char *cal = "n:gcal:///AGENDA?count=3";

    if (network_init() != FN_ERR_OK) return 1;

    /* Part II: an HTTPS API, via the JSON channel mode */
    network_open(api, OPEN_MODE_HTTP_GET, OPEN_TRANS_NONE);
    network_json_parse(api);
    n = network_json_query(api, "/number", (char *)buf);
    if (n > 0) printf("%s people are in space\n", buf);
    network_close(api);

    /* Part V: the next three things on the calendar */
    if (network_open(cal, 6, 0) == FN_ERR_OK) {
        while ((n = network_read(cal, buf, sizeof(buf) - 1)) > 0) {
            buf[n] = 0;
            printf("%s", buf);
        }
        network_close(cal);
    }
    return 0;
}
```

Listing 25. `whatsup.c` — compiles unchanged for every fujinet-lib platform; only the Makefile target differs.

THE FULL GUIDE

Writing Cross-Platform Client Apps Using fujinet-lib — `fujinet-manuals/writing-cross-platform-client-apps-using-fujinet-lib/` — the toolchain, the app skeleton, emulator testing, CI, and the API appendix; plus the runnable programs in `fujinet-lib-examples/network/`.

CHAPTER 38

IntyBASIC on the Intellivision

The Intellivision's FujiNet is a cartridge, and its bus is **shared memory**: a mailbox at `$9C00` that both the CP1610 and the FujiNet's RP2040 can read. A transaction is staged in RAM — device, command, parameters, payload — and submitted by bumping a sequence byte; the reply lands in a receive window. The `fujinet.bas` library wraps all of it in sixteen `GOSUB`s.

38.1 The transaction, honestly

```
stage:   mb_dev, mb_cmd, mb_nparam, parameters (fn_param),
         payload appended to the TX window (fn_putstr / fn_putnum)
submit:  SEQ = ACKSEQ + 1     — written LAST; this is the doorbell
wait:    poll ACKSEQ (the library gives it 900 frames)
verify:  reply code, error byte; results in the RX window
```

The command bytes are exactly Chapter 6's table — `$4F` opens, `$52` reads, `$53` statuses — aimed at network devices `$71`–`$78`. The library's shorthand does the staging for you; this is an open, verbatim from the guide's netcat:

```
    #fn_txlen = 0
    #fn_src = SC_URL : ls_max = 255 : GOSUB fn_strlen : GOSUB fn_putstr
    mb_dev = NET_DEVICEID
    mb_cmd = NETCMD_OPEN
    mb_nparam = 2
    pm_i = 0 : pm_size = 1 : #pm_val = OPEN_MODE_RW : GOSUB fn_param
    pm_i = 1 : pm_size = 1 : #pm_val = OPEN_TRANS_NONE : GOSUB fn_param
    GOSUB fn_transact
```

and the read loop, equally verbatim:

```
    GOSUB net_status
    IF #net_avail > 0 THEN
        #net_readlen = #net_avail
        IF #net_readlen > 64 THEN #net_readlen = 64
        GOSUB net_read
        IF fn_ok THEN
            FOR nc_i = 0 TO #net_gotlen - 1
                nc_c = PEEK(FN_RX + nc_i) AND 255
                GOSUB term_putc
            NEXT nc_i
        END IF
    END IF
```

For plain HTTP work there is `api_call`, the one-shot open→settle→read→ close that all three shipped games use.

38.2 The platform's sharp edges

- Derive `SEQ` from the **published** `ACKSEQ`, never a local counter, and write it last — the library does; hand-rolled transactions that don't will hang on the first exchange after a reset.

- For HTTP, the request fires at the first STATUS (Chapter 14′s lazy transaction), and `avail` counts **bytes so far** — the library's settle-poll waits for two consecutive equal STATUS reads before trusting it.
- Declare `ASM MEMATTR $8000, $9BFF, "+RWN"` and keep your variables below `$9C00`, or the emulator's mailbox is shadowed by your own RAM.
- IntyBASIC binds `=` tighter than `AND` — `(PEEK(FN_RX) AND 255) = 3` needs its parentheses.

THE FULL GUIDE

FujiNet Programming Guide for the Intellivision — `fujinet-manuals/intv/fujinet-programmers-guide/` — the complete mailbox map, every command with parameter tables, the full `fujinet.bas` listing, three networked games with their servers, and the netcat these excerpts came from.

CHAPTER 39

FujiNet NOS

NOS — the Network Operating System — is the odd one out: not a language but a **command line**, an Atari DOS whose disk is the network. Everything in Part II becomes a shell command; no program is written at all. It is also the fastest way to **exercise** this book's protocols by hand, which earns it the closing chapter.

39.1 The session

```
NCD N1:TNFS://fujinet.online/
DIR
LOAD GAMES/JUMPMAN.XEX
USER MOLLY
PASS SECRET
NCD N2:SMB://DEN-PC/ATARI/
NCOPY N1:DOCS/README,N2:BACKUP/README
TYPE N3:HTTPS://fujinet.online/
```

`NCD` mounts a devicespec on one of eight network drives — and because mounts live **inside the FujiNet** (Chapter 2), the drive stays mounted for any program you then run. `DIR`, `NCOPY` / `COPY`, `TYPE`, `DEL`, `RENAME`, `MKDIR`, `RMDIR`, `LOAD`, and `SAVE` do what forty years of DOS muscle memory expect, across TNFS, SMB, NFS, HTTP(S) and WebDAV, FTP, SD, and GDRIVE. `USER` / `PASS` are the credential specials of Chapter 2 wearing shell clothes — set them **before** the `NCD` that needs them. `NTRANS` is the sticky translation, and its golden rule is Chapter 3′s: `NTRANS N1: 0` before copying binaries.

Old programs come along free: a filespec of `D1:` – `D8:` is forwarded to `N1:` – `N8:`, so a 1982 BASIC program `SAVE`s onto a TNFS server without knowing networks exist.

39.2 The platform's sharp edges

- **N4: is NOS's own** — `NCOPY` builds network destinations and the `HELP` system fetches articles through it. Keep your mounts on other drives.
- Mounts are shared state: an `NCD` moves the drive for every program on the machine, and stays until unmounted (name the bare drive to unmount).
- The stream protocols — TCP, TELNET, SSH — are not shell material; NOS hands you to the programmer's guide for those.

> **THE FULL GUIDE**
> **NOS: An Introduction** — `fujinet-manuals/atari/nos_manual/` — the full command reference with every option, batch files and AUTORUN, the protocol capability tables, and the gentlest on-ramp to devicespecs ever printed for this hardware.

APPENDIX A

Error Codes at a Glance

The complete table, with meanings, is in Chapter 5; this is the wall-chart form, followed by where each platform surfaces the byte.

```
  1 SUCCESS                 165 INVALID_DEVICESPEC      205 ALREADY_IN_PROGRESS
131 WRITE_ONLY              166 INVALID_POINT           206 ADDRESS_IN_USE
132 INVALID_COMMAND         167 ACCESS_DENIED           207 NOT_CONNECTED
135 READ_ONLY               170 FILE_NOT_FOUND          208 SERVER_NOT_RUNNING
136 END_OF_FILE             200 CONNECTION_REFUSED      209 NO_CONNECTION_WAITING
138 GENERAL_TIMEOUT         201 NETWORK_UNREACHABLE     210 SERVICE_NOT_AVAILABLE
144 GENERAL                 202 SOCKET_TIMEOUT          211 CONNECTION_ABORTED
146 NOT_IMPLEMENTED         203 NETWORK_DOWN            212 BAD_USERNAME_PASSWORD
151 FILE_EXISTS             204 CONNECTION_RESET        213 COULD_NOT_PARSE_JSON
162 NO_SPACE_ON_DEVICE                                  214 CLIENT_GENERAL
                                                        215 SERVER_GENERAL
                                                        255 NO_BUFFERS
```

Environment	Where the code appears
Atari BASIC	`PEEK(749)` after `STATUS`; CIO errors also reach `TRAP` as BASIC errors
Applesoft	the third variable of `NSTATUS ch,bw,conn,err`; command-level failures via `ONERR` / `PEEK(222)`
ADAM SmartBASIC	BASIC error 37 raised (`?Network Error <n>`); `NSTATUS d,bw,conn,err` reads it back
C / fujinet-lib	`fn_device_error` after any call; data calls return the negative code directly
IntyBASIC	the fourth status byte in the RX window (`#net_err` from the library); link errors in `#mb_err`
NOS	printed as a message at the console

Table 26. Reading the error byte on each platform.

APPENDIX B

Protocol Capability Matrix

Protocol	R	W	A	RW	DIR	SEEK	REN	DEL	MKD	RMD	LCK	UNL	Port / notes
TNFS	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	16384 · anonymous
SD	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	–	–	local card
SMB	Y	Y	Y	Y	Y	Y	S	S	Y	Y	S	S	445 · URL or specials
NFS	Y	Y	Y	Y	Y	Y	S	S	Y	Y	S	S	2049 · UID/GID
FTP	Y	Y	–	–	Y	–	S	S	S	S	S	S	21 · anonymous only
SFTP	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	Y	22 · pass or key
HTTP/S	Y	Y	–	Y	Y	Y	Y	Y	Y	Y	S	S	80/443 · WebDAV ops
S3	Y	Y	Y	Y	Y	–	Y	Y	Y	Y	S	S	443 · keys or config
GDRIVE	Y	Y	Y	Y	Y	–	S	Y	Y	Y	S	S	OAuth (web UI)
ONEDRIVE	Y	Y	Y	Y	Y	–	Y	Y	Y	Y	S	S	OAuth (web UI)
TCP	Y	Y		Y	–	–	–	–	–	–	–	–	23 dflt · listen + accept
UDP	Y	Y		Y	–	–	–	–	–	–	–	–	port required
TELNET	Y	Y		Y	–	–	–	–	–	–	–	–	23 · negotiated
SSH	Y	Y		Y	–	–	–	–	–	–	–	–	22 · pass or key
WS/WSS	Y	Y		Y	–	–	–	–	–	–	–	–	80/443 · frames
SSH.KEYGEN	Y	–			–	–	–	–	–	–	–	–	report only
SSH.COPYID	Y	–			–	–	–	–	–	–	–	–	pass required
CLIPBOARD	Y	Y	Y	Y	–	–	–	–	–	–	–	–	index 0 writable
CPM	Y	Y		Y	–	–	–	–	–	–	–	–	console channel
GMAIL	Y	Y	–	–	Y	–	–	–	–	–	–	–	OAuth · W = compose/reply
IMAPS	Y	–	–	–	Y	–	–	–	–	–	–	–	993 · read-only
GCAL	Y	Y	–	Y	Y	–	–	–	–	–	–	–	OAuth · W = compose/edit
ICAL	Y	–	–	Y	Y	–	–	–	–	–	–	–	feeds · read-only
TEST	Y	Y	Y	Y	–	–	–	–	–	–	–	–	loopback

Y — implemented. **S** — accepted and reports success, but performs nothing (see Appendix E). **–** — refused or absent. Blank — the mode is accepted as its nearest neighbor (streams treat any mode as read/write; the utilities do their work at open regardless). R/W/A/RW/ DIR are aux1 4/8/9/12/6; SEEK–UNL are the specials `$25`, `$20`, `$21`, `$2A`, `$2B`, `$23`, `$24`. For mailbox and calendar protocols, DIR is the index listing and R spans counts, bodies, and details.

APPENDIX C

Scheme Quick Reference

Scheme	Ch.	Canonical form and query parameters
`TNFS`	8	`N:TNFS://host[:port]/path`
`SD`	9	`N:SD:/path`
`SMB`	10	`N:SMB://[user:pass@]server/share/path`
`NFS`	11	`N:NFS://host[:port]/export/path`
`FTP`	12	`N:FTP://host[:port]/path`
`SFTP`	13	`N:SFTP://user[:pass]@host[:port]/path`
`HTTP` `HTTPS`	14	`N:HTTPS://[user:pass@]host[:port]/path[?query]`
`S3`	15	`S3://[key:secret@]endpoint[:port]/bucket/key` · `?region=` `?tls=0|1`
`GDRIVE`	16	`GDRIVE:///path`
`ONEDRIVE`	17	`ONEDRIVE:///path`
`TCP`	18	`N:TCP://host[:port]/` · `N:TCP://:port/` (listen)
`UDP`	19	`N:UDP://[host]:port/`
`TELNET`	20	`N:TELNET://host[:port]/` · `?term=` `?cols=` `?rows=`
`SSH`	21	`N:SSH://user[:pass]@host[:port]/` · `?term=` `?cols=` `?rows=`
`WS` `WSS`	22	`N:WSS://host[:port]/path` · `?frame=text`
`SSH.KEYGEN`	23	`N:SSH.KEYGEN://ed25519/` · `?overwrite=1`
`SSH.COPYID`	24	`N:SSH.COPYID://user:pass@host[:port]/`
`CLIPBOARD`	25	`N:CLIPBOARD:///[0-9]` · `?binary=1`
`CPM`	26	`N:CPM://`
`TEST`	27	`N:TEST://anything/`
`GMAIL`	29	`GMAIL:///Folder[/N[/A]]` · `?range=a-b` `?newest=0|1` · mode 8: `/` compose, `/Folder/N` reply
`IMAPS`	30	`IMAPS://user:pass@host[:port]/FOLDER[/N[/A]]` · `?range=` `?newest=`
`GCAL`	32	`GCAL:///[sel]/VIEW[/DATE[/N]]` · `?category=` `?count=` `?days=` `?wkst=` `?tz=` · mode 8: `/[sel]` compose, `/…/N` edit
`ICAL` `WEBCAL` `ICALH`	33	`ICAL://feed-host/feed-path/VIEW[/DATE[/N]]` · same params as GCAL

APPENDIX D

Special Commands by Protocol

Chapter 6 holds the full table with semantics; this is the applicability map — which chapters' protocols answer which specials.

Specials	Honored by
`$20` – `$2B` rename/delete/lock/unlock/mkdir/rmdir	filesystem protocols (Part II), per the capability matrix
`$25` seek · `$26` tell	TNFS, SD, SMB, NFS, SFTP; HTTP on GET channels
`$2C` chdir · `$30` getcwd	filesystem protocols, via the per-bus prefix machinery
`$41` accept · `$63` close client	TCP in listening mode
`$44` set destination · `$72` get remote	UDP (`$72` on fujinet-pc builds only)
`$4D` HTTP channel mode	HTTP and HTTPS only
`$50` parse · `$51` query · `$FB` JSON parameters	any protocol, via the channel-mode machinery
`$54` translation · `$4C` set EOL · `$5A` interrupt rate · `$FC` channel mode	every channel — bus-level state
`$FD` username · `$FE` password	every channel; consumed by the protocols listed as credential-taking
`$FF` direction inquiry	every command — answered by the bus dispatcher itself

Table 27. Where each special lands.

APPENDIX E

Implementation Notes

The honesty appendix: rough edges in the firmware as of the colophon's commit, stated plainly so your program (or your patch) can account for them. None is dangerous; all are real.

1. **Windows would-block misreport.** On fujinet-pc Windows builds, a would-block socket condition falls through to the address-in-use case and reports 206 instead of success (`Protocol.cpp`).
2. **TCP close-client reports failure on success.** The `$63` special returns the error path even when the disconnect worked (`TCP.cpp`). Ignore its transaction status; trust the next status call.
3. **Silent stubs.** FTP's six filesystem specials, SMB's and NFS's rename/ delete/lock/unlock, and the base-class lock/unlock inherited by HTTP, S3, GDRIVE, and ONEDRIVE all report success while doing nothing. The capability matrix marks every instance `S` .
4. **HTTP `stat()` short-circuits** before its HEAD-request body runs (`HTTP.cpp`), so HTTP file sizes come from the transaction itself, not a probe.
5. **HTTP header reads lack a length clamp** — reading a collected header with a buffer shorter than the value copies past it (`HTTP.cpp`). Give header reads a 256-byte buffer and this cannot bite.
6. **One Telnet state machine.** The libtelnet handle is file-scope static (`Telnet.cpp`), so two simultaneous TELNET channels would share negotiation state. One BBS at a time.
7. **SSH status logic is inverted in name** (`isEOF` means "not EOF") and dereferences its channel unchecked (`SSH.cpp`); behavior is correct for a channel that opened successfully.
8. **`is_locked` means opposite things** — TNFS sets it for writable files, SFTP for read-only ones. Directory lock flags are decorative between protocols.
9. **Shadowed capability flags.** TNFS, FTP, SMB, and NFS redeclare the `*_implemented` flags in their own scope, shadowing the base copies — harmless today because nothing consults them (the dispatcher calls the virtuals directly), but a trap for future code that might.
10. **Error 211 is unreachable.** `CONNECTION_ABORTED` is defined and never assigned by any adapter.
11. **UDP multicast is scaffolding.** The address-class detection exists but nothing calls it; group membership is not implemented.
12. **The commit verdict is bus-dependent.** The mail and calendar commit-on-close verdict (Chapters 28 and 31) is latched through the close into the next status by the SIO, AdamNet, DriveWire, and RS-232 bus layers; the IEC, IWM, ComLynx, S100, and RC2014 layers do not yet latch it, so a failed send or commit may not surface there.
13. **Draft rejections are one number.** Every reason a mail or calendar draft can be refused — bad key, missing colon, bad time, missing required field, end before start, mixed date forms — reports the same 132; the specific cause is named only in the debug log.
14. **No delete, no attachments out.** The mail adapters never delete, move, or mark messages, and send `text/plain` only — no attachments on the way out. The calendar adapters compose and edit but cannot remove an event.

APPENDIX F

The Programmer's Guides

The per-platform chapters of Part VI are digests. The full guides — all in the `fujinet-manuals` repository unless noted, each with its complete device reference and a closing netcat — are:

Guide	Where and what
FujiNet Programming Guide for the Atari	`atari/programmers_guide/` — N: handler and BASIC, the XIO set, direct SIO for the network **and** Fuji devices, NDEV source, Action!, Logo
FujiNet BASIC Extension for the Apple II	`apple2/basic_extension/` — all eighteen commands, errors, memory map, full source
FujiNet Programming Guide for the Apple II	`apple2/programmers_guide/` — SmartPort from 6502 assembly; the command reference beneath the extension
FujiNet Programming Guide for the Coleco ADAM	`adam/programmers_guide/` — AdamNet transactions from EOS and CP/M, Z80 assembly, the network device at packet level
SmartBASIC 1.x FujiNet appendix	the `smartbasic-1.x` repository — every statement with examples; the patched interpreter itself
FujiNet Programming Guide for the Intellivision	`intv/fujinet-programmers-guide/` — the mailbox map, command tables, `fujinet.bas`, three complete networked games
Writing Cross-Platform Client Apps Using fujinet-lib	`writing-cross-platform-client-apps-using-fujinet-lib/` — the C toolchain end to end, from empty directory to CI-built binaries for five machines
NOS: An Introduction	`atari/nos_manual/` — the network as a DOS; every command, batch files, troubleshooting
FujiNet Programmer's Reference (MS-DOS)	`msdos/programmers_reference/` — the INT F5 interface for PC compatibles, with its own netcat and Mastodon reader

Table 28. Where to go when this handbook's chapter ends.

The habit this book hopes you have caught: the guides differ in **spelling**; the protocols never do. Anything Part II–V taught you is already true on every machine in this table.